SCIVELYX SUITE · ROCKFACE



RockFace e Python

Scripts, console, linha de comando e a API

A janela de scripts e o console, receitas conferidas para cada programa, o terminal, o Jupyter, os scripts de exemplo, a Mio escrevendo scripts e a referência completa da API.

Versão 0.31.0 • 2026-11-14

LCCMat · Universidade de Brasília • laribeirojr.github.io/rockface

Sumário

Como usar este manual 3

Começar 5

RockFace e Python 6

A janela de scripts e o console 8

Scripts em Python 10

Receitas 14

Receitas, programa por programa 15

A Mio escreve scripts 19

Os scripts de exemplo 21

Fora da interface 22

No terminal, no Jupyter e em outros programas 23

O RockFace como servidor MCP para outros assistentes de IA 25

Apêndices 27

Perguntas frequentes 28

Referência da API (em inglês, como no código) 29

Glossário 52

Como usar este manual

Este manual faz parte da coleção de manuais do RockFace Suite: um para cada programa (Petro, Atlas, Explorer, Lab e MC), um da **Mio**, a assistente, e um de **Python**, para quem quer automatizar o trabalho. O **Guia do usuário** (no site e em PDF) explica cada função em poucas páginas; os manuais vão além: contam o porquê de cada passo, mostram exemplos completos e listam **todos os campos, opções e botões** de cada tela, como o programa os mostra.

Como o manual está organizado

- **Parte I — Conhecendo o programa.** Para que ele serve, os conceitos que ele usa e a sua janela.
- **Parte II — Tutorial.** Um trabalho completo, do começo ao fim, com o projeto ILUSTRATIVO de exemplo. Faça-o uma vez com o programa aberto: em uma ou duas horas você terá usado tudo o que importa.
- **Parte III — Referência.** Um capítulo por função, sempre na mesma ordem:
  - **Para que serve** e **Antes de começar**;
  - **Passo a passo**, com os nomes dos botões como o programa os mostra;
  - **Opções** e **Como funciona** (o método, com a sua fonte na literatura);
  - **Exemplos práticos** e **Dicas**, escritos para este manual;
  - **Campos e opções da tela**: a tabela de cada campo — tipo, valores aceitos, intervalo, unidade, valor inicial e a dica que o programa mostra. Ela é lida do próprio programa, por isso é sempre igual ao que você vê;
  - **Por script**: as mesmas operações em Python (o manual de Python aprofunda);
  - **Limites e problemas comuns**.
- **Apêndices.** Todos os comandos com os seus atalhos de teclado, todos os campos de todas as abas, perguntas frequentes e glossário.

Convenções

- **Negrito** marca o que aparece na tela: botões, abas, campos (**Salvar**, **Composição**, **Pontos desejados**).
- `Texto em fonte fixa` é o que se digita: nomes de arquivos, comandos, código.
- Teclas: Ctrl+S, F1, Del. No macOS, Ctrl é ⌘.
- Os exemplos usam apenas dados **ILUSTRATIVOS** (inventados para o manual, com nomes marcados ILLUSTRATIVE). As imagens das telas vêm do programa real.

> **Dica:** o projeto de exemplo está sempre a um clique: na página inicial do Petro, **Abrir o exemplo ilustrativo** (uma lâmina com fotos, mapa mineral e análises pontuais) ou, na aba Ajuda, **Abrir a coleção ILUSTRATIVA** (48 descrições em quatro poços, para o Explorer e a Mio).

Onde mais procurar

- **F1** em qualquer programa abre o guia do usuário no idioma da interface, com busca.
- A **Mio** responde "como faço...?" com a página certa do guia (manual da Mio).
- O site https://laribeirojr.github.io/rockface/ tem o guia, os manuais e as novidades de cada versão.
- **Ajuda → Enviar opinião...** leva as suas sugestões e problemas à equipe.

Começar

RockFace e Python

Tudo o que se faz nas janelas do RockFace pode ser feito por um script em Python: abrir um projeto, ler e escrever descrições, contar, classificar, gerar relatórios, consultar a coleção, segmentar imagens, medir objetos, processar volumes de micro-CT, chamar a Mio e os conectores. A **API de scripts** (`rockface.api`, hoje na versão 1.23) é a mesma em todos os lugares: na janela de scripts do programa, no console, num terminal, num caderno Jupyter e no servidor MCP.

Por que usar scripts

- **Repetir sem erro.** O relatório de cada lâmina nova, a tabela do coordenador toda sexta, o LAS de cada poço: um script faz igual todas as vezes.
- **Fazer em lote.** Segmentar cem imagens, recalcular as classificações de um projeto inteiro, exportar todas as lâminas de um poço.
- **Ir além das janelas.** Juntar os números do RockFace com os seus próprios cálculos (NumPy, pandas, SciPy, matplotlib), ler planilhas de laboratório, gerar figuras sob medida.
- **Documentar o método.** Um script guardado no projeto é a receita do que foi feito, que qualquer colega pode ler e repetir.

Quatro maneiras de rodar

Onde	Para quê	Como
Janela de scripts	scripts do projeto, botões na faixa de opções	Petro → **Ferramentas → Python → Scripts** (Ctrl+Shift+P)
Console	experimentar uma linha, inspecionar o projeto	sob a saída da janela de scripts
Terminal	rodar sem a interface, agendar	`rockface script meu_script.py --project estudo.rockface -- arg1`
Jupyter ou qualquer Python	análises, figuras, cadernos	`import rockface.api as rf` · `rockface notebook estudo.rockface`

O que um script vê

Na janela de scripts, num script rodado por `rockface script` e num caderno gerado pelo RockFace, já existem:

Nome	O que é
`rf`	a API (`rockface.api`)
`project`	o projeto aberto (`rf.ProjectAPI`)
`current`	a descrição aberta na interface (ou `None`)
`ui`	a interface: `ui.message(texto)` escreve na barra de status; `ui.open(nome)` abre uma descrição ao fim
`args`	os argumentos dados depois de `--` na linha de comando

Fora do RockFace, abra o projeto você mesmo:

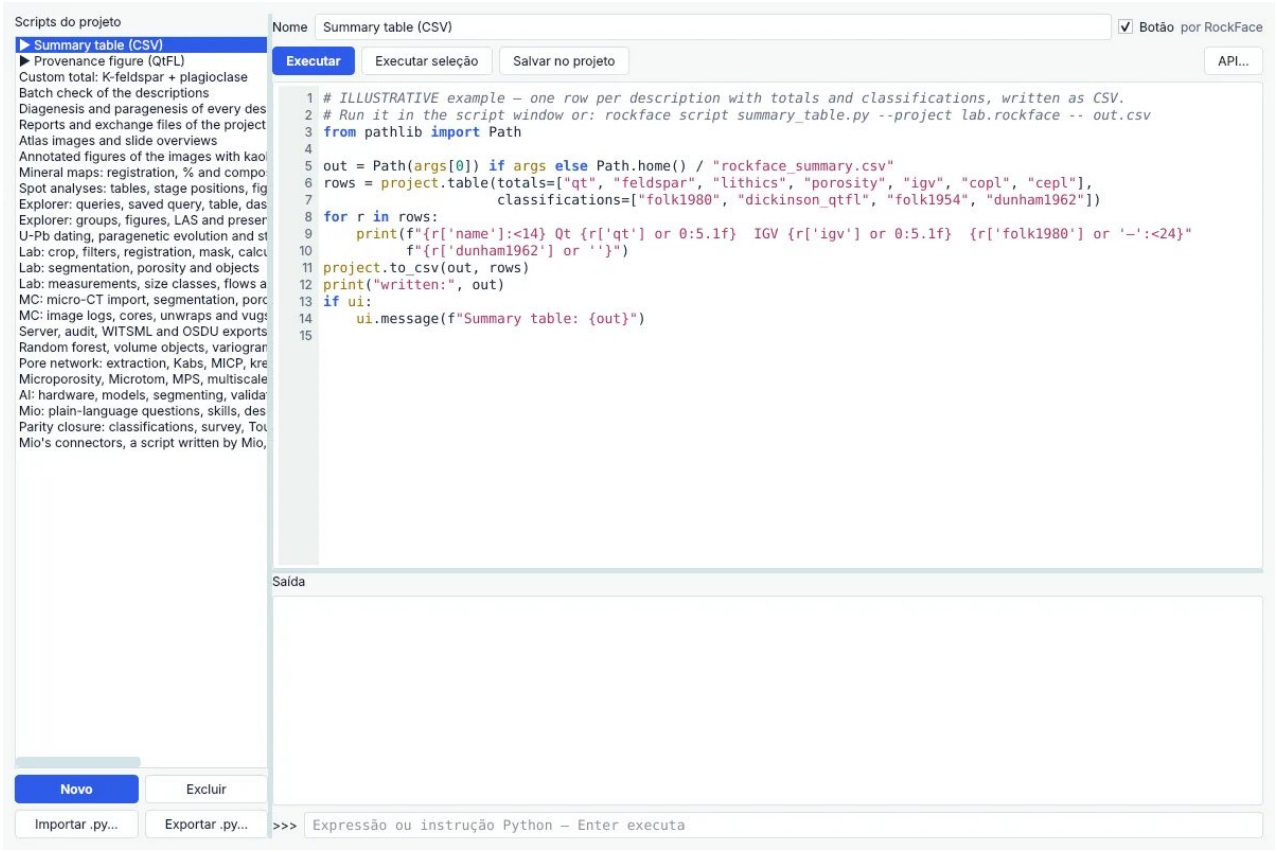
```
import rockface.api as rf

project = rf.open("estudo.rockface")          # um arquivo, ou o endereço do servidor da equipe
print(project)                                # <RockFace project 'estudo': 48 descriptions>
```

As regras da API

- **Nada muda sem `save`.** Ler é livre; alterar uma descrição exige `d.save()` (ou um método que diz que grava).
- **Os mesmos códigos da nomenclatura.** Constituintes, hábitos, localizações são os códigos da base de conhecimento (`kaolinite`, `booklet`, `ig_pore_filling`); `project.term("constituent", "kaolinite")` mostra o termo.
- **Os critérios do Explorer.** Toda seleção de amostras é uma lista de critérios `(conceito, atributo, valores)` ou `(conceito, atributo, mínimo, máximo)` — os mesmos que a Mio mostra em *Condições lidas*.
- **Auditoria.** O que um script grava fica na auditoria do projeto, com o nome do usuário, como na interface.
- **Exemplos ILUSTRATIVOS.** O projeto de exemplo (`rf.example(caminho)`) e a coleção (`rf.collection(caminho)`) têm dados inventados, para experimentar sem medo.

A janela de scripts e o console



A janela de scripts

Um passeio

1. Abra o exemplo ILUSTRATIVO no Petro e **Ferramentas → Python → Scripts** (Ctrl+Shift+P).
2. Em **Scripts do projeto**, à esquerda, estão os scripts de exemplo que vêm com o projeto — mais de vinte, um para cada parte do RockFace. Clique em **Summary table (CSV)**.
3. **Executar** (F5). A **Saída** mostra o que o script imprimiu; um erro aparece com a linha marcada no editor.
4. **Novo**, dê um **Nome**, escreva, **Salvar no projeto** (Ctrl+S). Marque **Botão** para que o script apareça sob o botão **Scripts** da faixa de opções.

O console

Sob a saída fica o **console**: uma linha de Python, executada com Enter, no mesmo espaço de nomes da última execução. É o lugar para experimentar antes de escrever o script. Com a coleção ILUSTRATIVA aberta:

```
>>> len(project.descriptions())
48
>>> arenitos = project.query(("identification", "facies", ["Sandstone"]))
>>> len(arenitos), [d.name for d in arenitos][:3]
(10, ['ILL-C003', 'ILL-C005', 'ILL-C012'])
>>> d = project["ILL-C003"]
>>> d.identification.well_name, d.identification.facies
('ILLUSTRATIVE-B1', 'Sandstone')
>>> t = d.totals(); round(t["porosity"], 1), round(t["qt"], 1), round(t["cement"], 1)
(25.0, 45.8, 16.7)
>>> d.classify("folk1980", label=True)
'Subarkose'
>>> header, rows = project.query_table("petrophysics", ("identification", "facies", ["Sandstone"]))
>>> header[:4], rows[0][:4]
(['Description', 'Porosity', 'Permeability', 'Grain density'], ['ILL-C003', '24.7', '1898', '2.789'])
>>> print(project.mio(lang="pt").ask("porosidade média por fácies").text)
Porosidade por fácies:
• Sandstone: média 19.64 (n = 10; 16.1–24.7)
• Boundstone: média 17.28 (n = 9; 3.5–25.9)
…
```

> **Dica:** `dir(d)` lista o que uma descrição sabe fazer; `help(d.totals)` mostra a explicação de um método. O console é o melhor lugar para descobrir a API.

Atalhos

Tecla	O que faz
F5 ou Ctrl+Shift+J	executa o script inteiro
Ctrl+J	executa a seleção, ou a linha do cursor
Ctrl+S	salva o script no projeto
Enter (no console)	executa a linha

Scripts como botões

Um script com **Botão** marcado aparece sob a seta do botão **Scripts** da faixa de opções do Petro, para toda a equipe que abre o projeto. Um clique o executa; se ele falhar, a janela de scripts abre com o erro. É a maneira de transformar uma tarefa da equipe em um comando do programa.

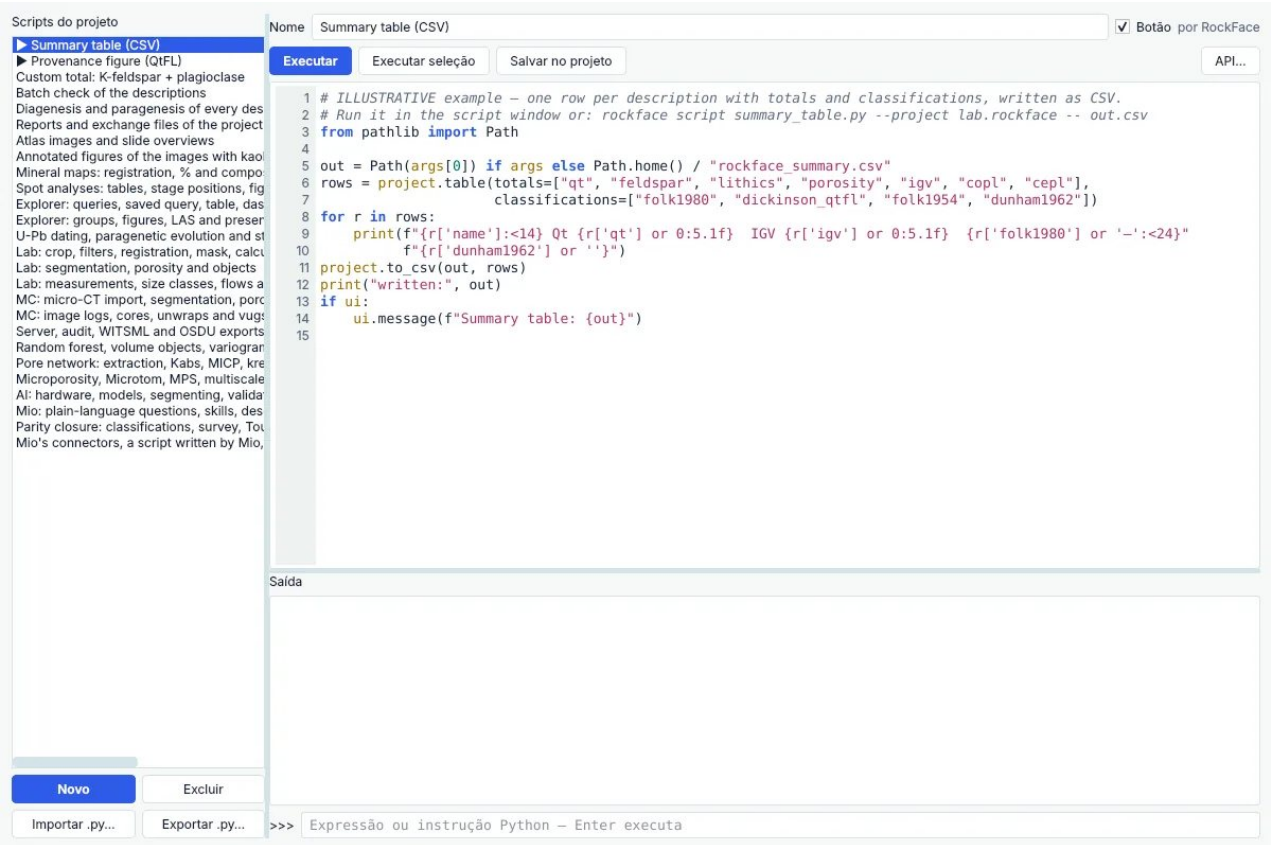
Antes de executar

Se a descrição aberta tem mudanças não salvas, o RockFace pergunta se deve salvá-la antes de executar o script, e a recarrega depois — o script e a janela nunca disputam a mesma descrição.

Scripts em Python

Para que serve

Todos os programas do RockFace podem ser conduzidos por scripts em Python por meio de uma API pública, `rockface.api`: para preencher muitas descrições de uma vez, gerar as tabelas e figuras de um estudo, conferir uma coleção inteira ou automatizar qualquer tarefa do Atlas, Explorer, Lab e MC. Os scripts ficam guardados no projeto, podem virar botões da faixa de opções e rodam também sem a interface, num terminal ou num notebook Jupyter.



No programa: Scripts em Python

Antes de começar

- Um projeto aberto, para guardar scripts nele (“Abra um projeto para guardar scripts nele.”).
- Python básico. O botão **API...** e o `docs/SCRIPTING.md` do código-fonte listam todas as chamadas.

Passo a passo

A janela de scripts

1. **Ferramentas → Python → Scripts** (Ctrl+Shift+P). A janela **Scripts Python** se abre.
2. **Scripts do projeto** à esquerda: **Novo**, **Excluir**, **Importar .py...**, **Exportar .py...**.
3. Dê ao script um **Nome**. Marque **Botão** para listá-lo sob a seta do botão **Scripts** da faixa de opções.
4. Escreva o script no editor (números de linha, realce de Python). Num script:
   - `rf` — a API; `project` — o projeto aberto; `current` — a descrição aberta na interface;
   - `ui.message(texto)` escreve na barra de status; `ui.open(nome)` abre uma descrição depois que o script termina.
5. **Executar** (F5 ou Ctrl+Shift+J) roda o script inteiro; **Executar seleção** (Ctrl+J) roda as linhas selecionadas ou a linha atual.
6. **Saída** mostra o que o script imprime e os seus erros, com a linha do erro marcada no editor ("O script parou com um erro na linha ...").
7. O **console** sob a saída avalia uma linha no mesmo espaço de nomes da última execução — Enter executa.
8. **Salvar no projeto** (Ctrl+S).

Se a descrição aberta tiver mudanças não salvas, o RockFace pergunta se deve salvá-la antes da execução, e a recarrega depois.

Um primeiro script

```
for d in project:
    t = d.totals()
    print(d.name, round(t["porosity"], 1), d.classify("folk1980", label=True))
rows = project.table(totals=["volume", "qt", "feldspar", "igv"], classifications=["folk1980"])
project.to_csv("resumo.csv", rows)
ui.message(f"{len(rows)} descrições gravadas")
```

Sem a interface

```
rockface script meu_script.py --project estudo.rockface -- arg1 arg2
```

No Jupyter ou em qualquer programa Python:

```
import rockface.api as rf
with rf.open("estudo.rockface") as prj:
    print(len(prj))
```

`rockface notebook estudo.rockface` grava um notebook que abre o projeto (página MC → Mais: copos, mapa de distâncias, vigia de pasta, reconstrução, transferência de escala e escoamento bifásico).

Os scripts de exemplo

O projeto de exemplo ILUSTRATIVO traz os scripts de exemplo: uma tabela-resumo, uma figura de proveniência, um total personalizado, uma conferência de todas as descrições, o resumo da diagênese, relatórios em lote, a visão geral do Atlas, figuras anotadas, mapas mineralógicos, análises pontuais, consultas e figuras do Explorer, datação e evolução, e os exemplos do Lab e do MC.

Opções

Opção	O que faz
Botão	Lista o script sob o botão Scripts da faixa de opções (aba Ferramentas).
Executar seleção	Roda só as linhas selecionadas, ou a linha atual.

Como funciona

- **As mesmas verificações.** Um script não consegue gravar o que os formulários recusariam: `d.save()` valida exatamente como os formulários e levanta `rf.ScriptError` com as mensagens, sem gravar nada.
- **Auditoria.** O que um script grava é registrado com o nome do usuário seguido de `[script]`.
- **Scripts escritos por outros.** Um script guardado num projeto e escrito por outra pessoa só roda depois de uma confirmação ("Um script pode ler e alterar o projeto e os arquivos deste computador. Executá-lo?").
- **Estabilidade.** `rf.API_VERSION` segue o versionamento semântico: dentro da 1.x os nomes só são acrescentados.
- **Licença.** Os scripts que usam a API são obra do próprio usuário: escrevê-los, executá-los e compartilhá-los é permitido.

Campos e opções da tela

Todos os campos desta tela como o programa os mostra (lidos do próprio programa, com o projeto ILUSTRATIVO aberto). O valor inicial é o do exemplo.

Campo	Tipo	Valores	Valor inicial	Observação
Saída	texto longo			
»>	texto	Expressão ou instrução Python — Enter executa		
Nome	texto			
Botão	caixa de marcação		marcada	Listar este script no botão Scripts da faixa de opções.

Botões e comandos

Comando	Onde	Observação
Executar	—	Ctrl+Shift+J, F5
Executar seleção	—	Ctrl+J

Comando	Onde	Observação
Salvar no projeto	—	Ctrl+S
API…	—	
Novo	—	
Excluir	—	
Importar .py…	—	
Exportar .py…	—	

Por script

Esta página é sobre scripts; a seção “Por script” de cada outra página deste guia mostra as chamadas daquela função.

Limites e problemas comuns

- “O script parou com um erro na linha …”: o traceback na saída mostra só as linhas do próprio script.
- Um script roda com os seus direitos no computador: execute só scripts em que confia.
- Um script longo bloqueia a janela enquanto roda; para cálculos longos do MC use trabalhos (página Trabalhos: computação em lote e remota).

Receitas

Receitas, programa por programa

As receitas abaixo são scripts curtos e completos. Todas rodam na sequência, no projeto de exemplo e na coleção ILUSTRATIVA — um teste automático do RockFace as executa a cada versão, de modo que o que está aqui funciona. Copie, troque os nomes pelos do seu projeto e use.

Comece abrindo os dois projetos ILUSTRATIVOS (numa pasta de trabalho):

```
import rockface.api as rf
from pathlib import Path

project = rf.example("exemplo.rockface")      # 5 descrições, imagens, mapa, pontos
col = rf.collection("colecao.rockface")        # 48 descrições de 4 poços
out = Path("saida")
out.mkdir(exist_ok=True)
```

Petro: descrições, totais e classificações

A tabela do coordenador. Totais e classificações de todas as descrições, em CSV (abre no Excel):

```
rows = project.table(totals=["volume", "qt", "feldspar", "cement", "igv"],
↪  classifications=["folk1980"])
project.to_csv(out / "resumo.csv", rows)
for r in rows:
    print(r["name"], round(r["qt"], 1), r["folk1980"])
```

Uma descrição por dentro. Identificação, totais, classificação e diagênese:

```
d = project["ILL-TS-001"]
print(d.identification.well_name, d.identification.top_depth)
t = d.totals()
print("porosidade", round(t["porosity"], 1), "IGV", round(t["igv"], 1))
print("Folk (1980):", d.classify("folk1980", label=True))
for code, env in d.interpret().items():
    print(env["status"], code, "regras:", ", ".join(env["rules"]))
for p in d.paragenesis():
    print("passo", p["step"], p["phase"])
```

Relatórios em lote. Um PDF por descrição, e um Word com todas:

```
for d in project:
    project.report(out / f"{d.name}.pdf", names=[d.name])
project.report(out / "todas.docx", lang="pt")
```

Atlas: imagens, legendas, mapas e pontos

As imagens da lâmina e as suas legendas automáticas.

```
for img in project.images():
    print(img.name, img.row.kind)
foto = project.image("Field B — kaolinite booklets")
print(foto.caption_auto())
```

Onde aparece a caulinita? A busca por conceito, nas anotações e nas descrições:

```
for img, onde in project.search_images(concept="kaolinite"):
    print(img.name, onde)
```

O mapa mineralógico contra a contagem de pontos.

```
mapa = project.image("ILLUSTRATIVE mineral map")
for r in mapa.compare(project["ILL-TS-001"]):
    print(f"{r['phase']:<35} mapa {r['map']:5.1f} %   contagem {r['count']:5.1f} %")
```

Os pontos de análise de uma lâmina.

```
for s in project["ILL-TS-001"].spots():
    print(s.name, s.kind, s.constituent, round(s.x_mm, 2), round(s.y_mm, 2))
```

Explorer: consultas, tabelas, figuras e LAS

Uma consulta, a sua tabela e a planilha.

```
calcita = ("diagenetic", "constituent", ["calcite_dia"])
porosa = ("petrophysics", "porosity", 15, None)
achadas = col.query(calcita, porosa)
print(len(achadas), "descrições:", [d.name for d in achadas][:5])
col.export_table(out / "calcita_porosa.xlsx", "petrophysics", calcita, porosa)
```

Os valores de um campo, com as contagens.

```
for poco, n in col.query_values("identification", "well"):
    print(poco, n)
```

Uma figura de revista, salva no projeto.

```
fig = col.figure(out / "phi_k.png", kind="crossplot", x="petrophysics.porosity",
                 y="petrophysics.permeability", log_y=True, colour_by="identification.facies",
                 preset="aapg-1col")
col.save_figure("Porosidade × permeabilidade", fig)
```

Um LAS por poço, para o Petrel.

```
arquivos = col.export_las(out / "las")
print([f.name for f in arquivos])
```

Lab: segmentar, separar, medir

```
pp = project.image("ILLUSTRATIVE mosaic PP")
poros = pp.segment("resin", name="PP poros")
print(f"porosidade de imagem {poros.porosity():.1f} %")
objetos = poros.objects(1, method="watershed", min_distance=6, min_px=20)
s = objetos.measure().summary("feret_max_um", exclude_edge=True)
print(s["n"], "poros; D50", round(s["d50_mm"], 3), "mm; seleção", round(s["sorting_phi"], 2), "φ")
```

O fluxo em todas as imagens de luz plana.

```
imagens = [i.name for i in project.images() if i.row.kind == "slide" and "PP" in i.name]
for r in project.run_flow(imagens, "pp"):
    print(r.sample["well"], r.sample["depth_m"], round(r.porosity, 1))
```

MC: um plugue digital

```
from rockface.synthetic import SyntheticVolume

d = project["ILL-TS-001"]
ct = d.import_volume(SyntheticVolume(120, 120, 80, voxel_um=5.0, blur=1.0, noise=8, plug=True),
                     name="ILLUSTRATIVE plug", voxel_um=5.0)
vol = ct.filter(("median", {"size": 3})).segment(roi="cylinder", name="poros")
print(f"porosidade {vol.porosity()['porosity']:.1f} %")
rede = vol.extract_network()
print(rede.stats()["pores"], "poros na rede")
```

Mio e conectores, por script

Perguntar à Mio e usar a resposta.

```
mio = col.mio(lang="pt")
r = mio.ask("amostras com calcita poiquilotópica e porosidade acima de 15")
print(r.text)
print(r.answer.names)                       # os nomes, para usar no script
```

Preparar uma consulta ao Crossref sem enviar nada.

```
col.enable_connector("crossref")
call = col.mio(lang="pt").ask("busque referências sobre cimentação por calcita em arenitos").answer
print(call.preview("pt"))                   # o que sairia; nada foi enviado
col.disable_connector("crossref")
```

Ao terminar

```
project.close()
col.close()
```

A Mio escreve scripts

Muitas tarefas de um estudo se repetem: o relatório de cada lâmina nova, a tabela de classificações para o coordenador, o LAS de cada poço para o Petrel, a apresentação da reunião. Para essas, peça um **script**: a Mio o escreve em Python, com as amostras da sua frase, e o abre na **janela de scripts**. Ela nunca o executa — você lê, muda o que quiser, executa (F5) e o guarda no projeto para a próxima vez.

Como pedir

Use a palavra *script* (ou *automatize*, *escreva um código*) e diga o que quer e de quais amostras:

- `escreva um script que gere o relatório em PDF de cada descrição com caulinita`
- `script que exporte as amostras com porosidade acima de 20 para uma planilha`
- `automatize a apresentação PowerPoint das amostras do poço ILLUSTRATIVE-A1`

Os detalhes da frase viram parâmetros: o formato (PDF, Word, Excel, HTML, CSV), o idioma do relatório, a classificação (Folk, McBride, Dickinson), *original* para a composição restaurada, e as amostras (as mesmas condições que a busca entende).

As receitas

Receita	Peça assim	O script
Relatório de cada descrição	*script que gere o relatório em PDF de cada descrição*	um arquivo por descrição, no formato pedido
Tabela de totais e classificações	*script com a tabela de totais e classificações Folk em Excel*	totais e classificações de todas as descrições
Resultado da consulta em planilha	*script que exporte as amostras com porosidade acima de 20 para uma planilha*	a tabela do que foi pedido (petrofísica, composição...)
Arquivo LAS de cada poço	*script que grave um arquivo LAS de cada poço*	um LAS 2.0 por poço
Páginas HTML	*script que gere páginas HTML das descrições*	um site estático das descrições
Cópia de segurança	*script para fazer cópia de segurança das descrições em XML*	os arquivos de intercâmbio
Apresentação	*script de uma apresentação PowerPoint das amostras com caulinita*	um .pptx da consulta
Diagrama ternário	*script do diagrama ternário Folk das amostras*	a figura, em PNG
Sequência paragenética	*script da sequência paragenética de cada descrição*	um gráfico por descrição
Contagem por poço e fácies	*script que conte as amostras por poço e fácies*	uma tabela de contagens
Valores faltantes	*script que liste as descrições sem permeabilidade*	a lista para completar
Fluxo do Lab	*script que rode o fluxo do Lab em todas as imagens*	porosidade e tamanhos de cada imagem

Receita	Peça assim	O script
Volumes de micro-CT	*script com a porosidade de cada volume de micro-CT*	a porosidade de cada volume segmentado
Imagens anotadas	*script que exporte as fotos com as anotações*	as imagens com as anotações desenhadas

Um exemplo completo

Pedido: *escreva um script que gere o relatório em PDF de cada descrição com caulinita*. A Mio responde "Script "Relatório de cada descrição" — amostras: diagenetic.constituent." e abre:

```
"""Relatório de cada descrição

Escrito pela Mio a partir do pedido: escreva um script que gere o relatório em PDF de cada descrição
↪  com caulinita
Leia, mude o que precisar e execute (Executar, ou F5). `project` é o projeto aberto; `rf` é a API de
↪  scripts.
"""

CRITERIA = [('diagenetic', 'constituent', ['kaolinite'])]   # the samples of the request (Explorer
↪  criteria)

from pathlib import Path

out = Path('reports')
out.mkdir(exist_ok=True)
FMT = 'pdf'                                          # pdf, docx, xlsx or html
done = 0
for d in project.query(*CRITERIA):                  # every description meant by the request
    safe = "".join(ch if ch.isalnum() or ch in "-_ " else "_" for ch in d.name)
    path = out / f"{safe}.{FMT}"
    project.report(path, names=[d.name], lang='')
    done += 1
    print("written", path)
print(done, "reports in", out.resolve())
```

Repare em `CRITERIA`: é a sua frase, em critérios do Explorer. Para outro conjunto de amostras, troque a linha — `[('identification', 'well', ['ILLUSTRATIVE-A1'])]`, `[('petrophysics', 'porosity', 20.0, None)]` — ou peça de novo.

Do script ao botão

1. Na janela de scripts, dê um **Nome** ao script e marque **Botão**.
2. **Salvar no projeto** (Ctrl+S).
3. O script aparece sob a seta do botão **Scripts** da faixa de opções do Petro: um clique roda a tarefa, para toda a equipe que abre o projeto.

> **Dica:** peça o script mais próximo do que você quer e ajuste à mão. Um script de 15 linhas que você entende vale mais que um de 150 que ninguém lê. O manual **RockFace e Python** mostra a API inteira.

Os scripts de exemplo

O RockFace traz um script de exemplo para cada parte do programa. Eles estão no projeto de exemplo (janela de scripts) e na pasta `rockface/example_scripts` da instalação, e rodam no projeto de exemplo. São o melhor ponto de partida: copie o mais próximo do que você quer e mude.

Script	O que mostra
Summary table (CSV)	totais e classificações de todas as descrições num CSV
Provenance figure (QtFL)	o diagrama de proveniência de Dickinson
Custom total: K-feldspar + plagioclase	um total do projeto, definido por script
Batch check of the descriptions	a conferência de todas as descrições
Diagenesis and paragenesis of every description	ambientes, regras e sequência paragenética
Reports and exchange files of the project	relatórios em lote e arquivos de intercâmbio
Atlas images and slide overviews	imagens, escalas e visões gerais das lâminas
Annotated figures of the images with kaolinite	figuras anotadas a partir da busca por conceito
Mineral maps: registration, % and composites	registro, quantificação e imagem composta
Spot analyses: tables, stage positions, figure	pontos, referenciais de instrumento e figura
Explorer: queries, saved query, table, dashboard	consultas, consultas salvas e painel
Explorer: groups, figures, LAS and presentation	grupos, figuras de revista, LAS e PowerPoint
U-Pb dating, paragenetic evolution and stratigraphy	idades, evolução e coluna estratigráfica
Lab: crop, filters, registration, mask, calculator	a preparação das imagens
Lab: segmentation, porosity and objects	segmentação e objetos
Lab: measurements, size classes, flows and LAS	medidas, classes, fluxos e LAS por profundidade
MC: micro-CT import, segmentation, porosity, 2D in 3D	volumes e a lâmina dentro do volume
MC: image logs, cores, unwraps and vugs	perfis de imagem e testemunhos
Server, audit, WITSML and OSDU exports	modo servidor e exportações corporativas
Random forest, volume objects, variogram, rotation	floresta aleatória e ferramentas de volume
Pore network: extraction, Kabs, MICP, krel, production	a rede de poros e as simulações
Microporosity, Microtom, MPS, multiscale network, jobs	microporosidade e multiescala
AI: hardware, models, segmenting, validation, fine-tuning	RockFace IA por script
Mio: plain-language questions, skills, description suggestions	a Mio por script
Parity closure: classifications, survey, Touchstone table, publication, 3D composite	as funções da 0.28
Mio's connectors, a script written by Mio, the MCP server	conectores e servidor MCP

> **Dica:** os nomes dos scripts e os comentários estão em inglês, como o código; as mensagens que eles mostram na interface seguem a língua do programa.

Fora da interface

No terminal, no Jupyter e em outros programas

Um script sem a interface

```
rockface script meu_script.py --project estudo.rockface -- saida/ 2026
```

O projeto é aberto como `project`; o que vem depois de `--` chega ao script como a lista `args`. Sem a interface, `ui` é `None` — escreva `if ui: ui.message(...)` nos scripts que rodam dos dois jeitos.

Os scripts de exemplo que vêm com o RockFace rodam assim, por exemplo:

```
rockface example exemplo.rockface
rockface script $(python -c "import rockface.example_scripts as e; print(e.HERE /
↪  'summary_table.py')") \
    --project exemplo.rockface -- resumo.csv
```

Agendar uma tarefa

Um script no terminal pode ser agendado pelo sistema: o **Agendador de Tarefas** do Windows, o `cron` do Linux, o `launchd` do macOS. Por exemplo, todo dia às 7h, o relatório das descrições salvas na véspera, ou a publicação do projeto no ambiente corporativo (`rockface publish estudo.rockface --if-due`).

Jupyter

```
rockface notebook estudo.rockface --open
```

grava, ao lado do projeto, um caderno Jupyter que já abre o projeto com a API, e o abre no Jupyter quando ele está instalado. Num caderno, combine o RockFace com o pandas (`pip install pandas`) e o matplotlib:

```
import pandas as pd
import rockface.api as rf

project = rf.open("estudo.rockface")
header, rows = project.query_table("petrophysics")
df = pd.DataFrame(rows, columns=header)
df["Porosity"] = pd.to_numeric(df["Porosity"], errors="coerce")
df.describe()
```

Qualquer programa Python

A API é um pacote Python comum: `pip install rockface` (ou o instalador do RockFace) e `import rockface.api as rf`. Um serviço web interno, um robô de pesquisa, um pipeline de dados podem ler e escrever projetos RockFace — inclusive o banco do servidor da equipe (`rf.connect(endereço)`), com as permissões do usuário.

Os comandos do terminal

Comando	O que faz
`rockface petro`, `atlas`, `explorer`, `lab`, `mc`	abre um programa (com um projeto, se dado)
`rockface example arquivo` · `rockface collection arquivo`	grava o projeto de exemplo ou a coleção ILUSTRATIVA
`rockface script arquivo.py --project p -- args`	roda um script sem a interface
`rockface notebook projeto`	grava um caderno Jupyter do projeto
`rockface llm setup` · `info` · `use` · `ask projeto "pergunta"`	o modelo de linguagem local e a Mio no terminal
`rockface ai setup` · `info` · `models`	o hardware de IA e os modelos
`rockface connectors list` · `enable` · `disable` · `test` · `tools` · `secret` · `call` · `confidential`	os conectores da Mio
`rockface mcp serve projeto` · `config projeto`	o RockFace como servidor MCP
`rockface server …`	o modo servidor: iniciar, usuários, publicar, cópia, auditoria, travas
`rockface export witsml` · `osdu`	poços em WITSML 1.4.1.1, manifesto OSDU
`rockface publish projeto`	publica no ambiente corporativo, numa periodicidade
`rockface feedback`	o formulário de opinião dos testadores

`rockface <comando> --help` mostra as opções de cada um.

Outros assistentes de IA

O servidor MCP (`rockface mcp serve`) dá a um assistente de IA as ferramentas `ask`, `search`, `describe`, `table`, `guide`, `write_script` e `api`, e o recurso `rockface://api` com esta referência. Com `--allow-scripts`, também `run_script`. Veja o manual da **Mio**, capítulo *A Mio fora do RockFace*.

O RockFace como servidor MCP para outros assistentes de IA

Para que serve

Além de chamar serviços externos, o RockFace pode ser chamado por eles. O comando `rockface mcp serve` abre um projeto como **servidor MCP** (Model Context Protocol): um assistente de IA que fala MCP — o Claude Desktop, um editor de código, um agente corporativo — passa a consultar as amostras, ler tabelas, achar páginas do guia e escrever scripts para o RockFace, conversando em linguagem natural com o usuário. É o caminho para tarefas maiores que as receitas da Mio: o assistente lê a referência da interface de programação e escreve o script, que você executa no RockFace.

Antes de começar

- O RockFace instalado no computador e um projeto.
- Um assistente que aceite servidores MCP. Você é responsável pelo que o assistente envia para fora: se ele roda na nuvem, as respostas do RockFace chegam ao serviço dele. Para um projeto confidencial use um assistente local, ou não use este recurso.

Passo a passo

Ligar o RockFace ao assistente

1. Gere a configuração: `rockface mcp config projeto.rockface`. Sai um trecho JSON `mcpServers` com o comando que inicia o servidor.
2. Cole esse trecho no arquivo de configuração MCP do assistente (no Claude Desktop, o arquivo de configuração das ferramentas de desenvolvedor) e reinicie o assistente.
3. Pergunte ao assistente, por exemplo:
    - "No RockFace, quantas amostras têm calcita e quais são os poços?"
    - "Mostre a tabela de petrofísica das amostras com porosidade acima de 15 %."
    - "Como faço a contagem de pontos com platina motorizada no RockFace?"
    - "Escreva um script RockFace que, para cada poço, exporte o LAS e uma tabela de classificações de Folk."

Deixar o assistente executar scripts

Com `rockface mcp config projeto.rockface --allow-scripts` o servidor oferece também `run_script`: o assistente roda scripts no projeto e vê a saída. Use só num projeto de trabalho, com cópia de segurança, e com um assistente em que confie.

Servir por HTTP

`rockface mcp serve projeto.rockface --http 8765` serve o mesmo conteúdo por HTTP (Streamable HTTP) em `127.0.0.1:8765`, só neste computador.

Opções

Ferramenta do servidor	O que dá ao assistente
`ask`	a resposta da Mio a uma pergunta (contagens, listas, estatísticas, valores, lacunas, uma descrição)
`search`	as descrições de uma pergunta e as condições lidas
`describe`	uma descrição em palavras
`table`	um conceito em tabela (identificação, composição, petrofísica, totais, classificações...), opcionalmente só das descrições de uma pergunta
`guide`	as páginas deste guia que respondem a uma pergunta
`write_script`	um script escrito pelas receitas da Mio (devolvido, não executado)
`api`	a referência da interface de programação (`rf`)
`run_script`	só com `--allow-scripts`: executa um script no projeto

Recursos: `rockface://api` (a referência) e `rockface://guide/<página>` (as páginas do guia). Opções do comando: `--lang pt|en|es` (idioma das respostas), `--allow-scripts`, `--http PORTA`.

Como funciona

O servidor usa o SDK oficial do MCP em Python (licença MIT). Cada chamada do assistente gera uma linha na auditoria do projeto (`mcp`), com a ferramenta e o texto pedido. As ferramentas, exceto `run_script`, só leem o projeto. As respostas vêm do banco de dados e das receitas testadas; o assistente pode reformulá-las, mas os números são os do RockFace.

Por script

```
cfg = rf.mcp_config("projeto.rockface")          # o trecho mcpServers
print(cfg)
print(rf.api_reference()[:2000])                 # o que a ferramenta api devolve
```

Limites e problemas comuns

- O assistente não vê o RockFace: confira o caminho do projeto no trecho `mcpServers` e reinicie o assistente.
- Respostas em inglês: inicie com `--lang pt`.
- `run_script` não aparece: falta `--allow-scripts`.
- O projeto está aberto no RockFace ao mesmo tempo: as leituras funcionam; gravações de `run_script` e do programa podem se cruzar — feche um deles para gravar.

Apêndices

Perguntas frequentes

Preciso saber Python? Para rodar os scripts de exemplo e os que a Mio escreve, não: basta ler e mudar um nome de arquivo ou uma condição. Para escrever os seus, o básico de Python (variáveis, `for`, `if`, listas e dicionários) basta para quase tudo.

Um script pode estragar o projeto? Um script pode gravar, como a interface. Antes de rodar um script novo num projeto importante, **Salvar projeto como...** uma cópia, ou rode-o no projeto de exemplo. A auditoria registra o que foi gravado.

Onde os arquivos são gravados? Caminhos relativos (`"saida/resumo.csv"`) partem da pasta em que o RockFace foi aberto — na janela de scripts, normalmente a pasta do usuário. Use caminhos absolutos ou `Path.home() / "…"` quando precisar saber exatamente onde.

Posso usar outras bibliotecas? Sim: tudo o que estiver instalado no Python do RockFace. NumPy, SciPy, scikit-image e matplotlib vêm com a instalação completa; outras (pandas, por exemplo) se instalam com `pip` no mesmo Python.

Como descubro o nome de um constituinte? `project.term("constituent", "kaolinite")`, ou a janela da nomenclatura no Petro (**Ferramentas → Nomenclatura**), que mostra os códigos. A Mio também diz: *o que é caulinita?*

O script funciona no meu projeto e não no do colega. Os nomes (poços, fácies, constituintes do projeto) podem ser diferentes. Escreva os nomes uma vez no começo do script, como variáveis, para que o colega mude só ali.

A API muda entre versões? A versão da API (`rf.API_VERSION`, hoje 1.23) só cresce: o que existe continua a existir. A referência completa está no apêndice e em `rockface://api` pelo servidor MCP.

Como rodo um script num banco do servidor? `rf.connect("postgresql+psycopg://usuario@servidor:5432/rockfa` (o endereço que o administrador der). As permissões são as do usuário.

Referência da API (em inglês, como no código)

A lista completa das funções da API, versão por versão, como está em docs/SCRIPTING.md. Os nomes e as explicações estão em inglês, a língua do código.

Version 1.0 came with RockFace 0.7. Version 1.1, with 0.8, added the diagenesis functions. Version 1.2, with 0.9, added reports and exchange files. Version 1.3, with 0.10, added the images of RockFace Atlas. API 1.4, with 0.11, added the annotations of the images, their automatic captions and the search of images by concept. API 1.5, with 0.12, added mineral maps, their registration on the optical image, composites and quantification. API 1.6, with 0.13, added spot analyses and the instrument frames of a section. API 1.7, with 0.14, added the queries, saved queries, result tables and dashboard of RockFace Explorer. API 1.8, with 0.15, added groups of descriptions on the ternary classifications, the figure generator, PowerPoint presentations and the LAS export of the wells. API 1.9, with 0.16, added U-Pb dating on the Tera-Wasserburg diagram, the paragenetic evolution of several samples and the stratigraphic column with the ages. API 1.10, with 0.17, added RockFace Lab: crop, filters, sample mask, registration of one image on another, resampling and the image calculator. API 1.11, with 0.18, added the segmentation of RockFace Lab: segmentation maps and their porosity, editing of maps in place with undo, and objects (grains, pore bodies) with their editing. API 1.12, with 0.19, added the measurements of the objects, size classes and distributions, tables with formulas and filters, charts, the flows over several images with LAS by depth, the whole section block by block, and the exchange of label images and NetCDF. API 1.13, with 0.20, added RockFace MC: micro-CT volumes of a sample, imported from the usual formats, cropped, filtered, resampled, combined by formulas, segmented with their porosity, connectivity and two-point correlation, exported, registered, and a thin-section image located in a volume. API 1.14, with 0.21, added the well logs of RockFace MC (image logs, core unwraps and photographs, curves and tables by depth), their formats, processing, inpainting, segmentation, planar features and pore sizes, and whole cores processed, oriented, unwrapped and laid on an image log. API 1.15, with 0.22, added the server mode (connecting to a server database, users and levels, the audit, edit locks, backups and publishing) and the WITSML and OSDU exports. API 1.16, with 0.23, added the random forest (pictures and volumes) and the Bayesian method on volumes, the models kept in the project, 3D painting and the effects of a segment editor on volumes, the objects of the pore space, anisotropic diffusion, shading correction, registration with rotation, the variogram, the REV and their report, the attributes and the PCR file, the transitions between the phases of a map, and the editing of log segmentations and log objects. API 1.17, with 0.24, added the pore networks (extraction, the Statoil files, absolute permeability, mercury intrusion, relative permeability, sensitivity tests, production, Kabs REV, the report) and the open rock data (public micro-CT images downloaded and imported). API 1.18, with 0.25, added the porosity maps and their Darcy permeability, the Microtom simulations and the lattice-Boltzmann permeability, MPS image generation, the two-scale pore network, saving into an existing NetCDF file, the textured 3D surfaces, the Geolog package, batch and remote jobs and the permeability model of image logs. API 1.19, with 0.26, added RockFace AI: the hardware and the options, the model hub (models shipped, in a hub folder and in the project; import, export, publication, lineage), pictures, whole sections, volumes and image logs segmented by models, training and fine-tuning with metrics on held-out data, validation, the unsupervised segmentation, the instance segmenter of image logs, the generative model of a volume (SinGAN) and the spot cycle. API 1.20, with 0.27, added Mio: questions in plain language answered by the database (L1), Mio's tools and skills, the local language model (catalogue, recommendation for the hardware, llama.cpp or a local server, download) and a segmentation map turned into description suggestions. API 1.21, with 0.28, added the parity

closure: McBride and Bramkamp & Powers with the reservoir-rock symbol, packing classes, the files of the reference programs, lab reports in PDF, the scale bar of a picture, the 3D composite and the periodic publication (L13), the well trajectory, plug-ins (also from Git), the sample catalogue, the Touchstone-ready table, XML by mapping, the tools of the GeoSlicer source audit, two-phase flow with LBPM and a map aligned on a segmentation without landmarks. API 1.22, with 0.29, added the analyses and analysis requests of a sample (made in 0.3, now also from scripts) and the testers' feedback form. API 1.23, with 0.30, added Mio's connectors (outside services through MCP or directly, switched on per project, every call confirmed and audited), the scripts Mio writes for repetitive tasks and RockFace as an MCP server for other AI assistants.

Projects

Call	What it does
`rf.open(path)`	Opens a `.rockface` project. It is a context manager: `with rf.open(p) as prj:`.
`rf.create(path, overwrite=False)`	Creates an empty project.
`rf.example(path)`	Writes and opens the ILLUSTRATIVE example project.
`for d in prj`, `len(prj)`, `prj["name"]`, `prj.find(name)`	Iterate, count and find descriptions. Templates are left out; use `prj.descriptions(templates=True)` to include them.
`prj.new(name, **identification)`	Creates a description, e.g. `prj.new("TS-9", well_name="W-1", top_depth=2501.3)`.
`prj.copy(name, new_name, template=False)`, `prj.delete(name)`	Copy or delete a description.
`prj.search(text, category=None)`, `prj.term(category, code)`, `prj.kb`	The knowledge base. Search works in any language, accents ignored.
`prj.add_total(code, label, rf.Selector(...), …)`, `prj.custom_totals`	Custom totals of the project. They are validated, and a duplicate code raises `ScriptError`.
`prj.custom_schemes = [rf.Scheme(...)]`	Custom ternary classifications, built from `rf.Pole` and `rf.Field`.
`prj.table(totals=[…], classifications=[…], original=False, names=None)`	One dict per description.
`prj.to_csv(path, rows)`	Writes rows as UTF-8 CSV.
`prj.schemes()`, `prj.total_codes()`	Codes of every classification and total.
`prj.scripts()`	The scripts kept in the project.

Descriptions

Call	What it does
`d.identification`, `d.macro`, `d.micro`, `d.pores`, `d.composition`, `d.count`, `d.classification`, `d.carbonate`	The records, i.e. the dataclasses of `rockface.core`. Change their fields, then call `d.save()`.
`d.add_line(constituent, points=0, percent=None, nominal="", **attributes)`	Adds a composition line. Attributes are `location`, `modifier`, `habit`, `relation`, `related` and `related_location`.
`d.lines()`	Composition lines as dicts, with the computed percentage in `share`.
`d.check()`	Every message the save would give, without writing.
`d.save()`	Validates exactly as the forms do. Any error raises `rf.ScriptError` with `.messages`, and nothing is written.
`d.update(**identification)`	Changes identification fields and saves.
`d.reload()`	Forgets unsaved changes.
`d.totals(original=False)`	Every total, in % of the rock, plus IGV, COPL, CEPL and ICOMPACT.
`d.classify(code, original=False, label=False)`	The field or class of a classification, or `None` without data. `KeyError` for an unknown code.
`d.point(code, original=False)`	The normalised (a, b, c) of a ternary classification.
`d.summary()`	Totals and classifications, present and original.

Diagenesis (API 1.1, RockFace 0.8)

Call	What it does
`d.interpret(all_environments=False)`	Diagenetic environments: code → `{"status", "auto", "regime", "rules"}`. The status is the petrographer's decision (`confirmed`, `rejected`) when there is one, else `interpreted` or `evidence`.
`d.environments(label=False)`	Environments interpreted or confirmed.
`d.paragenesis()`	The paragenetic sequence in order: `uid`, `phase`, `kind` (precipitation or dissolution), `step`, `regime`, `rules`.
`d.diagenesis`	The record: `decisions[env] = "confirmed" | "rejected"`, `notes`, `env_notes`, and the sequence set by hand (`order`, `regimes`). Save with `d.save()`.
`prj.table(…, environments=True)`	Adds the interpreted or confirmed environments to each row.
`rf.paragenesis_figure(path, d)`	The paragenetic chart as PNG or SVG, without the interface.
`rf.knowledge_graph(path=None, environment="", highlight=())`	The knowledge graph of the rules (nodes and edges); written as DOT, SVG or PNG when a path is given.
`rf.dg.RULES`, `rf.dg.ENVIRONMENTS`	The rules and environments, with their sources.

Reports and exchange files (API 1.2, RockFace 0.9)

Call	What it does
`d.report(path, lang="", **options)`	The report of one description. The format follows the suffix: `.pdf`, `.docx`, `.xlsx` or `.html`. Unsaved changes are not in it: call `d.save()` first.
`prj.report(path, names=None, lang="", **options)`	Several descriptions in one file (all of them when `names` is `None`).
`prj.html_pages(folder, names=None, lang="", **options)`	One HTML page per description and an `index.html` that links them; returns the index.
`prj.export(path, names=None, attachments=True, templates=False, images=False)`, `d.export(path)`	Descriptions to a RockFace exchange file (`.rfx`, XML): identification, every part, and the analyses, requests and pictures of their samples; with `images=True` (API 1.3), the Atlas images of their thin sections too.
`prj.export_folder(folder, names=None, …)`	One exchange file per description.
`prj.import_file(paths, names=None, policy="rename")`	Import from files or folders. `policy` for names already used: `rename` (adds " (2)"), `skip` or `replace`. Returns one dict per description: `name`, `action`, `as`, `messages`, `file`.
`rf.read_exchange(path)`	Reads an exchange file without importing it: `.names()`, `.descriptions`, `.samples`, `.custom_totals`.
`rf.reports`	The report module: `rf.reports.Options`, `rf.reports.build(prj.raw, d.id)` (the blocks), `rf.reports.write(reports, path)`.

Report options: `sections` (any of `identification`, `macro`, `micro`, `pores`, `composition`, `totals`, `classification`, `diagenesis`, `analyses`, `requests`, `pictures`), `figure_mm` (width of the pictures), `ternary_mm` (width of the ternary diagrams), `ternaries` (their codes; empty means every one with a point), `figures` (`"inline"` or `"end"`) and `original` (also the original composition). The language of a report does not depend on the interface's.

```
with rf.open("lab.rockface") as prj:
    prj.report("well-1.pdf", lang="pt", sections=["identification", "composition", "classification"])
    prj.html_pages("site", lang="en")
    prj.export("for-the-lab.rfx", names=["TS-001", "TS-002"])
with rf.create("copy.rockface") as other:
    for r in other.import_file("for-the-lab.rfx", policy="skip"):
        print(r["name"], r["action"])
```

RockFace Atlas images (API 1.3, RockFace 0.10)

Images belong to the thin section of a description: whole-slide images (`kind="slide"`) and detail photos (`kind="photo"`) placed at their position in the section frame (mm, the frame of the point count). Each is kept in the project as a pyramid of tiles, so any part of a multi-gigapixel image is read at once at any resolution.

Call	What it does
`d.add_image(path, kind="slide", category=None, modality="pp", name="", caption="", magnification=None, um_per_px=None, …)`	Adds an image (PNG, JPEG, TIFF, BigTIFF, OME-TIFF; scanner formats with OpenSlide installed) to the description's thin section. The scale comes from the file, else from `um_per_px`, else from the objective table. Returns an image (`img`).
`d.add_photo(path, x_mm=None, y_mm=None, magnification=None, modality="pp", stage_x_mm=None, stage_y_mm=None, …)`	A detail photo at its position, with the stage position at which it was taken.
`d.images(kind=None)` (`"slide"`, `"photo"`, `"map"`), `prj.images(text="", kind=None, category=None, modality=None, magnification=None, well=None)`, `prj.image(name)`	The library: images found by any of these filters.
`img.name`, `img.kind`, `img.category`, `img.modality`, `img.magnification`, `img.um_per_px`, `img.width`, `img.height`, `img.levels`, `img.x_mm`, `img.y_mm`, `img.size_mm`	What the image is.
`img.region(x, y, w, h, level=0)`	Pixels (NumPy RGB array) of a rectangle given in full-resolution pixels, at the resolution of `level` (each level halves the previous).
`img.render(width, box=None)`, `img.export(path, max_side=8000, box=None, annotated=False, caption_file=False, lang=None)`	A Pillow image, or a PNG, JPEG or TIFF file, of the whole image or of `box`; with `annotated=True` (API 1.4) its annotations are drawn, and `caption_file=True` writes the automatic caption beside it (`.txt`).
`img.overview(path=None, width=2000, photos=True)`	A whole slide with the fields of its detail photos numbered and a scale bar.
`img.to_mm(px, py)`, `img.to_px(x_mm, y_mm)`, `img.footprint()`	Between pixels and the section frame; the rectangle (mm) of a photo.
`img.update(**fields)`, `img.calibrate(p1, p2, distance_um)`, `img.delete()`	Changes are checked as in the window; an error raises `ScriptError`.
`img.retake(path, **changes)`, `img.versions()`	A new photo of the same field (same position, stage position and objective), linked to the first; all photos of the field, oldest first.
`prj.objectives = {"10": 0.65, "20": 0.325}`	µm per pixel of the camera for each objective.
`rf.atlas`	The Atlas core module (`rf.atlas.ArraySource(array)` adds an image from a NumPy array).

```python
with rf.open("lab.rockface") as prj:
    d = prj["TS-001"]
    slide = d.add_image("TS-001_PP.tif", category="scanner", modality="pp")
    photo = d.add_photo("field1.jpg", x_mm=4.2, y_mm=3.6, magnification=10, stage_x_mm=31.2,
↪   stage_y_mm=24.6)
    slide.overview("TS-001_fields.png", width=2400)
    crop = slide.region(*slide.to_px(4.0, 3.4), 400, 300)            # NumPy array, full resolution
```

Annotations, captions and the search by concept (API 1.4, RockFace 0.11)

An annotation is an arrow, rectangle, ellipse, polygon or text label drawn on an image, or the image's note. Each can carry *features*: a constituent of the nomenclature with its location, habit, modifier and relation, as in a line of the composition. Points are full-resolution pixels. Annotations that say something (a feature or a text) are lettered A, B, … in the order they were drawn; the letters are drawn on the image and used by the caption.

Call	What it does
`img.annotations()`	The annotations, in order, each an `rf.Annotation` (`id`, `kind`, `points`, `text`, `features`, `colour`, `width`, `font`, `fill`, `letter`).
`img.annotate(kind, points, text="", features=(), colour="#FFD25A", width=3.0, font=14.0, fill=0.15)`	Adds one: `kind` is `arrow` (tail, head), `rect` or `ellipse` (two corners), `polygon` (three or more vertices) or `label` (one point and a text). Features are `rf.Feature(constituent, location="", habit="", modifier="", relation="", related="", related_location="")` or dictionaries; unknown codes raise `ScriptError`.
`img.note(text, features=())`	The general annotation of the image: one per image, replaced by each call.
`img.update_annotation(id, **changes), img.delete_annotation(id)`	Changes are checked as in the window.
`img.caption_auto(lang=None)`	The automatic caption: name, kind and light, objective, sample, depth and thin section, the position in the section, the note, "(A) ...; (B) ..." and the scale.
`img.render_annotated(width, box=None)`	A Pillow image with the annotations drawn.
`prj.search_images(rock_type=None, concept=None, attribute=None, value=None, where="both")`	The hierarchical search: images of thin sections of a rock type (`("folk1980", "subarkose")` or `rf.RockType`), showing a concept (a constituent or a group of the nomenclature, e.g. `"clay_dia"`) with an attribute (`location`, `habit`, `modifier`, `relation`, `related`, `related_location`) of a value. `where` is `"annotations"`, `"descriptions"` (the composition of the thin section) or `"both"`. Returns `[(img, reasons)]`, the reasons being the letters, `note` or `description`.
`prj.hierarchy(where="both")`	The tree rock type → concept → attribute → value → image ids, as in the "By concept" tab.
`prj.export_images(folder, images=None, annotated=True, max_side=4000, fmt="png", caption_file=True, lang=None)`	Every image (or those given) as files named after the image (`_annotated` when drawn), each with its caption.
`rf.an`	The annotations core module.

```
with rf.open("lab.rockface") as prj:
    img = prj.image("Field B — kaolinite booklets")
    img.annotate("rect", [[820, 410], [1260, 760]], features=[rf.Feature("kaolinite",
↪   habit="booklet")])
    img.annotate("label", [[1300, 420]], text="Kln")
    img.note("Kaolinite booklets fill the pores.", [{"constituent": "kaolinite"}])
    print(img.caption_auto("en"))
    for found, reasons in prj.search_images(concept="kaolinite", attribute="habit", value="booklet"):
        found.export(f"{found.name}.png", max_side=3000, annotated=True, caption_file=True)
```

Mineral maps, registration and quantification (API 1.5, RockFace 0.12)

A mineral map (from QEMSCAN, MLA, TIMA, a µXRF phase map or any classified image) is an image of kind `map` whose pixels are *phases*. Each phase (`rf.Phase`) has a name, a colour, concepts of the nomenclature (constituent, group or kind codes: `["quartz", "quartz_dia"]`, `["pore"]`), a role (`mineral`, `pore`, `background`, `unclassified`) and a visible flag. Any image can be *registered* on the section frame with landmark pairs; the map is then drawn over every slide of the section, and its regions are those of the section.

Call	What it does
`d.add_map(path, legend=None, name="", caption="", um_per_px=None, tolerance=12)`	Adds a map: a palette image, a grey-level image of phase numbers (8 or 16 bits) or an RGB image of phase colours. The legend is a CSV/TXT file (with a header such as `Mineral,R,G,B` or `index,name,colour`, or without one), a 3D Slicer colour table (`index name R G B A`), legend text or a list of entries; by default the legend file beside the map. Colours of an RGB map are matched to the legend within `tolerance`. Names are matched to concepts and roles. Returns an image (`img`).
`d.maps()`	The maps of the description's thin section.
`img.phases()`, `img.set_phase(i, name=…, colour=…, concepts=…, role=…, visible=…)`, `img.set_phases(list)`, `img.merge_phases(keep, others)`	The phases, by index; changes are checked as in the window.
`img.labels(x, y, w, h, level=0)`	Phase indices (NumPy `uint8`) of a rectangle of a map.
`img.register(pairs, model="affine", target=None)`	Registers the image with landmark pairs: `(pixel here, (x_mm, y_mm))`, or `(pixel here, pixel of target)` with a calibrated `target` image. `model` is `"similarity"` (≥ 2 pairs: rotation, scale, shift) or `"affine"` (≥ 3 pairs). Returns the fit: `rms_um`, `loo_um` (leave-one-out error: each pair predicted by the others), `residuals_um`, `um_per_px`, `rotation_deg`.
`img.registration`, `img.unregister()`	The stored registration (model, matrix, pairs, errors) or `{}`; removing it places the image by its scale again.
`img.quantify(region=None, level=0)`	Pixels, area (mm²) and % of every phase over the whole map or a region of the section frame: `("rect", (x0, y0, x1, y1))`, `("polygon", [(x, y), …])` in mm, or an annotation (`rf.Annotation` or `("annotation", id)`) of any calibrated image. Background and unclassified pixels are left out of the %. Returns rows (`index`, `name`, `colour`, `role`, `concepts`, `pixels`, `area_mm2`, `percent`), `area_mm2`, `porosity()`.
`img.by_concept(region=None)`	The % gathered by concept of the nomenclature.
`img.compare(description, region=None)`	The map % of every phase next to the % of the composition lines of a description that match its concepts: `[{"phase", "concepts", "label", "map", "count", "diff"}]`.
`img.composite(path=None, base=None, add_to_library=False, name="", width=2400, opacity=0.55, mode="blend", hidden=(), box=None, legend=True, scale_bar=True)`	The optical slide `base` (by default the plane-polarized slide of the section) with the map blended over it or its phase outlines (`mode="outline"`), with a legend of the % shown and a scale bar; written to `path` and returned as a Pillow image. With `add_to_library`, it is also added to the library in the frame of the base (`img.added`).
`rf.mineral_map`	The mineral-map core module (`read_map`, `read_legend`, `fit`, `MapData`, `CompositeOptions`, …).

```python
with rf.open("lab.rockface") as prj:
    d = prj["TS-001"]
    qem = d.add_map("TS-001_qemscan.tif", legend="TS-001_qemscan.csv", um_per_px=10)
    pp = prj.image("TS-001 PP")
    fit = qem.register([((140, 120), (380, 288)), ((1250, 90), (1860, 314)), ((700, 520), (1102, 854)),
                        ((180, 880), (390, 1304))], model="similarity", target=pp)
    print(f"RMS {fit.rms_um:.1f} µm, leave-one-out {fit.loo_um:.1f} µm")
    for row in qem.compare(d):
        print(row["phase"], round(row["map"], 1), row["count"])
    qem.composite("TS-001_composite.png", base=pp, opacity=0.5)
```

Spot analyses and instrument frames (API 1.6, RockFace 0.13)

A spot analysis (`rf.Spot`) belongs to the thin section and is kept in the section frame (mm), so it is drawn at the same place on every image of the section. It has a kind (`geochemistry`, `dating`, `fluid_inclusion`, `other`), a name unique in the section, `x_mm`, `y_mm`, `size_um`, `method`, `laboratory`, `date` (YYYY-MM-DD), `analyst`, the mineral analysed (`constituent`, `location` of the nomenclature),

`photo_id` (the detail photo of the area), `meta` (free metadata), `notes`, `source` (where an imported spot came from) and `values`: `rf.Value(name, value, error, unit, level="2s", below_dl=False)`. An *instrument frame* ties the coordinates of a laser, microprobe or microscope stage to the section frame with fiducial marks read in both.

Call	What it does
`d.spots(kind=None)`, `d.spot(name)`	The spots of the description's thin section.
`d.add_spot(kind, x_mm, y_mm, name="", values=(), size_um=…, method=…, laboratory=…, date=…, analyst=…, constituent=…, location=…, photo=…, meta={…}, notes=…)`	Adds a spot (the name is given in order, G01, D01, FI01…, when left out). Values are `rf.Value`, tuples `(name, value, error, unit, level, below_dl)` or dicts. Errors raise `ScriptError`; implausible values (a negative ppm, rho outside −1…1) only warn.
`d.update_spot(spot, **changes)`, `d.delete_spot(spot)`	By `rf.Spot`, name or id.
`rf.spots.derived(spot)`	Values derived from the measured ones, each with its source: the salinity of a fluid inclusion from Tm ice (Bodnar 1993) and the $^{206}Pb/^{238}U$ age from $^{238}U/^{206}Pb$ (λ^{238} of Jaffey et al. 1971, no common-Pb correction). Shown, never stored.
`d.import_spots(path, coords="mm", image=None, frame=None, kind="geochemistry", skip_errors=True)`	A laboratory table (CSV): a spot name, x and y — in the section frame (`"mm"`, `"um"`), in pixels of a calibrated `image` (`"px"`) or in an instrument `frame` (`"frame"`) — and value columns: `Sr (ppm)` with its unit, followed by `±`, `2s`, `err` or named `Sr 2s`, `Sr err 1s` for the uncertainty; `<0.5` is below the detection limit; text columns become metadata. Rows with errors are left out.
`d.export_spots(path, kind=None)`	One row per spot, one column per value with its uncertainty and level.
`d.calibrate_frame(name, marks, model="similarity", unit="mm")`	An instrument frame from fiducial marks `((u, v) on the instrument, (x_mm, y_mm) in the section)`; `unit` of the instrument `"mm"` or `"um"`. Returns the fit (`rms_um`, `loo_um`, `residuals_um`). A frame of the same name is replaced (calibrate again after the slide is put back).
`d.frames()`, `d.to_frame(name, x_mm, y_mm)`, `d.from_frame(name, u, v)`	Where a point of the section is on an instrument, and back: to find a spot again.
`d.spot_figure(path=None, base=None, count=True, width=2400, kinds=…, labels=True, legend=True, box=None, map_opacity=0.5)`	The section with the registered mineral map, the counted points of this description and the spots, labelled, with a legend.
`prj.spots(kind=None, constituent="", method="", value=None)`	Spots of the whole project; `constituent` may be a group (`"quartz"`); `value=("Th", 100, None)` keeps spots whose Th is at least 100.
`rf.spots`	The spot-analysis core module (`Spot`, `Value`, `read_csv`, `figure`, `salinity_bodnar`, `age_206_238`…).

```
with rf.open("lab.rockface") as prj:
    d = prj["TS-001"]
    d.calibrate_frame("LA-ICP-MS stage", [((41250, 18630), (1.0, 1.0)), ((57240, 18965), (17.0, 1.0)),
                                          ((41020, 29380), (1.0, 11.75))], unit="um")
    d.import_spots("TS-001_U-Pb.csv", coords="frame", frame="LA-ICP-MS stage", kind="dating")
    fi = d.add_spot("fluid_inclusion", 4.184, 3.488, values=[rf.Value("Th", 118.5, 1.0, "°C", "1s"),
                                                             rf.Value("Tm ice", -12.3, 0.2, "°C", "1s")],
                    method="microthermometry", constituent="quartz_dia", meta={"type": "primary"})
    print(rf.spots.derived(fi))                          # salinity 16.24 wt% NaCl eq. (Bodnar 1993)
    print(d.to_frame("LA-ICP-MS stage", fi.x_mm, fi.y_mm))
    d.spot_figure("TS-001_spots.png")
```

RockFace Explorer: queries and dashboard (API 1.7, RockFace 0.14)

Every description is read into *facts*, by concept (`rf.explorer.CONCEPTS`): `identification` (well, field, basin, formation, zone, facies, rock type, depths, sample type, uses…), `macroscopic`, `microscopic`, `primary`, `diagenetic` and `pore_composition` (one row per composition line, with its share %),

`pore_system`, `petrophysics`, `analyses`, `classifications`, `totals`, `interpretation`, `images`, `spots` (one row per spot, with its values and derived values) and `maps` (one row per phase). A description meets a query when, for each concept named, one and the same row meets all the criteria of that concept, and every concept is met.

Call	What it does
`prj.query(*criteria, **identification)`	The descriptions meeting every criterion. A criterion is `rf.Criterion(concept, attribute, values)` or `rf.Criterion(concept, attribute, low=…, high=…)`, a tuple `(concept, attribute, value or values)` or `(concept, attribute, low, high)` (an end may be `None`), or a saved `rf.Query`. Keywords are the simple query on identification: `well="X"`, `facies=["A", "B"]`, `top_depth=(4100, 4200)`.
`prj.query_table(concept, *criteria, labels=True, every_row=False, **identification)`	`(header, rows)` of a concept for the descriptions found: the rows that meet the criteria (`every_row`: every row of those descriptions); labels as in the interface, or codes and numbers with `labels=False`.
`prj.query_values(concept, attribute, *criteria, **identification)`	`[(value, number of descriptions)]` of an attribute among the descriptions found.
`prj.save_query(name, *criteria, **identification)`, `prj.saved_queries()`, `prj.delete_query(name)`	The queries kept in the project (the Explorer's Save / Open query).
`prj.export_table(path, concept, *criteria, **identification)`	The result table as XLSX or CSV.
`prj.dashboard(*criteria, field=…, formation=…, zone=…, well=…, facies=…, image_category=…)`	The data of the dashboard: `.facies()` (samples by facies), `.phi_k()` (porosity %, permeability mD, facies, id), `.grain_density(bins)`, `.depths()` (measured, corrected and adjusted depth), `.sample_types()`, `.gallery("mosaic_pp" | "mosaic_xp" | "scanner" | "micro" | "mineralogy")` (image ids).
`prj.explorer`	The `rf.explorer.Explorer` of the project (facts read again after every write): `.attributes(concept)`, `.values(…)`, `.match(query)`, `.table(…)`, `.facts`.
`rf.collection(path, n=48, images=True)`	Writes and opens the ILLUSTRATIVE collection: four wells in two fields, two formations, three zones, five facies, compositions (barite in some), petrophysics, U-Pb spots and small images.
`rf.explorer`, `rf.Criterion(…)`, `rf.Query(criteria, name)`	The Explorer core module and its query classes.

```
with rf.open("collection.rockface") as prj:
    print([d.name for d in prj.query(("diagenetic", "constituent", "barite"))])     # which samples
    ↪ have barite?
    print([d.name for d in prj.query(("spots", "system", "U-Pb"), well="ILLUSTRATIVE-A2")])  # dated,
    ↪ in well A2
    good = prj.query(("petrophysics", "porosity", 15, None), ("petrophysics", "permeability", 100,
↪   None))
    prj.save_query("Good reservoir", ("petrophysics", "porosity", 15, None))
    header, rows = prj.query_table("petrophysics", facies="Grainstone")
    board = prj.dashboard(field="Example Field A")
    print(board.facies(), board.grain_density(10))
```

Groups, figures, presentations and LAS (API 1.8, RockFace 0.15)

A *group* (`rf.Group(name, query, ids=[], marker="circle", fill="#2F5BEA", edge="#15243A", size=1.0, visible=True)`) is a set of descriptions: those found by `query` (a saved query's name or an `rf.Query`) plus the ids picked by hand. Up to ten groups are drawn together on a ternary classification. Markers: `rf.groups.MARKERS`. Groups may also be given as tuples `(name, criteria)`, e.g. `("Well A", [("identification", "well", "A")])`; they then get the marker and colours of their position.

A *figure* is an `rf.FigureSpec(kind, x, y, colour_by, groups, query, scheme, original, log_x, log_y, bins, depth, title, x_label, y_label, names, legend, preset, width_mm, height_mm, font_size, lang)`. Kinds: `crossplot`, `histogram`, `bar` (number of samples per value), `box` (an attribute by category), `depth` (an attribute against depth, one panel per well), `ternary` (groups or categories on a classification). Attributes are written `"concept.attribute"` (`"petrophysics.porosity"`, `"identification.facies"`, `"totals.qt"`). Where a description has several rows of a concept, the mean is used, except that `x` and `y` of the same concept give one point per row. Journal sizes: `rf.figures.PRESETS`.

Call	What it does
`prj.group_points(scheme, groups, original=False)`	Each group on a ternary classification (`"folk1980"`, `"dickinson_qtfl"`, `"dickinson_qmflt"` or a custom one), recomputed from the saved compositions. Returns one result per group: `.points` holds (id, name, (a, b, c), field code) and `.missing` holds the names without a point.
`prj.save_groups(name, groups, scheme="", original=False)`, `prj.saved_groups()`	Group sets kept in the project (the Groups tab's *Save groups* / *Open groups*).
`prj.figure(path=None, spec=None, where=(), dpi=600, **fields)`	Builds a figure from `spec` (an `rf.FigureSpec` or a saved figure's name) or from keyword fields (`kind="crossplot", x=…, y=…, log_y=True, preset="elsevier-1col"`). `where` takes criteria that restrict the descriptions. With `path`, the figure is written as PNG, TIFF, JPEG, SVG or PDF, the plotted data as CSV, or a SciVelyx Graphs project (`.scivelyx.json`, the same figure with its data, to finish it there). Returns the spec.
`prj.figure_data(spec)`	`(header, rows)` of the points of a figure.
`prj.save_figure(name, spec)`, `prj.saved_figures()`, `prj.delete_figure(name)`	Figures kept in the project, drawn again from the latest records.
`prj.export_las(folder, *criteria, curves=…, depth="top_depth", company="", **identification)`	Writes one LAS 2.0 file per well of the descriptions found, with an irregular depth step (`STEP 0`) and `NULL -999.25`. `curves` are `"concept.attribute"` codes or `rf.Curve(code, mnemonic, unit, description, discrete)`. Text attributes become discrete curves of integer codes, with the code table in the Parameter and Other sections. Two samples at the same depth are merged. Defaults: `rf.las.DEFAULT_CURVES`.
`prj.presentation(path, *criteria, slides=None, figures_=(), concept="identification", gallery="mosaic_pp", title="", lang="", **identification)`	Writes a 16:9 PowerPoint presentation (.pptx) that opens title-first, with the query's criteria. The other slides are either `slides` (`{"kind": "figure", "figure": spec}`, `{"kind": "table", "concept": c}`, `{"kind": "gallery", "gallery": g}`, `{"kind": "criteria"}`) or, when `slides` is not given, the `figures_`, the table of `concept` and the image gallery.
`rf.figures`, `rf.groups`, `rf.las`, `rf.presentation`	The core modules (`rf.las.read(path)` reads a LAS 2.0 file back; `rf.presentation.Deck` builds slides one by one).

```
with rf.open("collection.rockface") as prj:
    wells = [rf.Group(w, rf.Query([rf.Criterion("identification", "well", [w])])) for w in ("A1",
↪   "A2")]
    prj.figure("qfr.pdf", kind="ternary", scheme="folk1980", groups=wells, colour_by="groups")
    phik = prj.figure("phi_k.png", kind="crossplot", x="petrophysics.porosity",
↪   y="petrophysics.permeability",
                      log_y=True, colour_by="identification.facies", preset="elsevier-1col")
    prj.figure("phi_k.scivelyx.json", phik)                      # open it in SciVelyx Graphs
    prj.export_las("las", curves=["petrophysics.porosity", rf.Curve("totals.qt", "QT", "%")])
    prj.presentation("barite.pptx", ("diagenetic", "constituent", "barite"), figures_=[phik])
```

U-Pb dating, paragenetic evolution and stratigraphy (API 1.9, RockFace 0.16)

The U-Pb spots (spot analyses of kind `dating`, or with `system` "U-Pb", that have $^{238}U/^{206}Pb$ and $^{207}Pb/^{206}Pb$) of the descriptions found are dated together on the Tera-Wasserburg diagram: York

regression (York et al. 2004), lower intercept with the concordia, MSWD and probability of fit, free or anchored on an initial $^{207}Pb/^{206}Pb$ (a value or Stacey & Kramers 1975 common lead). Errors in the results are 1σ (`s_age`) and the 95 % interval (`conf95`). The method and its check against published values are in DATING_EVOLUTION.md.

Call	What it does
`prj.upb_points(*criteria, constituent="", exclude=(), **identification)`	The Tera-Wasserburg points (`rf.geochron.Point`: name "description/spot", x, sx, y, sy, rho, included) of the U-Pb spots of the descriptions found; `constituent` keeps one mineral (or group); names in `exclude` are left out of the regression.
`prj.date(*criteria, constituent="", anchor="", anchor_value=None, anchor_error=0.01, exclude=(), figure=None, **identification)`	The age (`rf.geochron.Dating`: `age`, `s_age`, `age_2s`, `conf95`, `y0`, `s_y0`, `mswd`, `p`, `n`, `fit`, `notes`). `anchor`: `""` (free), `"stacey_kramers"` or `"value"` (`anchor_value` ± `anchor_error`, relative). With `figure`, the Tera-Wasserburg diagram is written (PNG, SVG, PDF).
`prj.save_age(name, *criteria, constituent="", habit="", anchor="", …, notes="")`, `prj.ages()`, `prj.delete_age(name)`	Ages kept in the project with the phase they date (`rf.dating.Age`); they are placed in the evolution and in the stratigraphic column.
`prj.evolution(*criteria, path=None, photos=True, title="", lang="", **identification)`	The paragenetic evolution of the descriptions found (all when no criteria): `.phases` (`key`, `event`, `label`, `regime`, `water`, `samples`, `shares`, `share_mean`, `ages`, `photos`) and `.conflicts`. With `path`, the chart is written (PNG, SVG, PDF).
`prj.set_phase(key, water=None, regime=None)`	Sets by hand the water source (`rf.evolution.WATERS`) or the regime (`eo`, `meso`, `telo`) of a phase, by its key `"constituent|habit|kind"`; `""` clears.
`prj.stratigraphy`, `prj.stratigraphy = rows`	The formations, top to base: `[{"name", "top", "base", "colour"}]`, with their depositional ages in Ma (base ≥ top).
`prj.strat_column(path, *criteria, title="", lang="", **identification)`	The stratigraphic column with the depositional ages and the U-Pb ages of the samples found (PNG, SVG, PDF).
`rf.geochron`, `rf.dating`, `rf.evolution`	The core modules: `rf.geochron.york(x, sx, y, sy, rho)` (any straight line with errors in both coordinates), `rf.geochron.date(points, anchor)`, `rf.geochron.concordia_tw(t)`, `rf.geochron.stacey_kramers(t)`.

```python
with rf.open("collection.rockface") as prj:
    d = prj.date(formation="Example Formation", constituent="calcite_dia", anchor="stacey_kramers",
                 figure="tw.pdf")
    print(f"{d.age:.1f} ± {d.age_2s:.1f} Ma (2σ), MSWD {d.mswd:.2f}, n = {d.n}")
    prj.save_age("Sandstone calcite", ("identification", "formation", "Example Formation"),
                 constituent="calcite_dia", habit="poikilotopic", anchor="stacey_kramers")
    evo = prj.evolution(formation="Example Formation", path="evolution.png")
    prj.set_phase(evo.phases[-1].key, water="hydrothermal")
    prj.strat_column("column.pdf")
```

RockFace Lab: crop, filters, sample mask, registration, calculator (API 1.10, RockFace 0.17)

Every call writes a **new image** of the section (the original is never changed); the new image keeps the section frame and records in `img.history` what was done to which image. Rectangles are given in mm of the section frame (`(x0, y0, x1, y1)` or `rf.Roi`), or in the pixels of an image (`box = (x, y, w, h)`). Filter steps are names of `rf.lab.FILTERS` with their parameters: `"equalize"`, `("gaussian", {"sigma": 2})`, `rf.Step("shading", {"reference": "resin"})`. The methods and their checks are in LAB.md.

Call	What it does
`prj.crop(images, roi=None, box=None, reference=None, suffix=" (crop)")`	Crops several images of a section with one rectangle (`roi` in mm, the name of a saved region, or `box` in the pixels of `reference`); each image to the part it covers. Returns the new images.
`prj.rois`, `prj.save_roi(name, roi)`, `prj.delete_roi(name)`	Rectangles saved in the project.
`img.crop(roi=None, box=None, name="")`	Crops one image; it keeps its place in the section.
`img.filter(steps, name="")`	Writes the filtered image (block by block, the same as filtering the whole image at once).
`img.filter_preview(steps, width=1200, box=None)`	The filtered image (or `box`) as a NumPy array, for a look; nothing is written.
`img.sample_mask(split=False, n_largest=0, resin="colour", samples=None, save=True, name="", gap=0.01, smooth=0.01, min_area=0.001)`	The useful area of the section (rock without glass and borders), optionally split into fragments by the resin wider than `gap` (fraction of the shorter side), keeping the `n_largest`; `resin="samples"` with `samples=(rock pixels, resin pixels)`. Kept as a mask map (returned), or returned as `rf.lab.MaskResult` with `save=False`.
`img.register_on(fixed, pairs=None, model="similarity", auto=True, search="", save=True)`	Aligns this image on `fixed`: landmark `pairs` [((x, y) here, (x, y) on fixed)] fitted with `model` ("translation", "similarity", "affine", "tps") and/or automatically (ECC of the grain outlines, from the section frames or from the pairs; `search` "shift", "rotation" or "full"). Returns `rf.lab.Transform` (`matrix`, `rotation_deg`, `scale`, `ecc`, `rms`, `loo`); saved as the image's registration unless `save=False`.
`img.resample_on(fixed, name="")`	This image written on the pixel grid of `fixed` (its saved registration, else the frames).
`img.areas_mm2()`	{phase: area in mm²} of a mask or map.
`img.history`	What Lab did to make the image: `op` (crop, filter, mask, resample, calculate), `source`, parameters, `registration`.
`prj.calculate(formula, reference, images=None, name="", output="auto", scale=None)`	Image calculator over the images of the section of `reference`: `{name}` or a name of `images`, with a channel `.r .g .b .l .h .s .v`; numbers, `+ - * / **`, comparisons, `& | ~`, `abs sqrt log log10 exp min max where clip round`. A comparison gives a mask (map), a number a grey image (clipped to 0–255, or `scale=(low, high)` stretched).
`rf.lab`, `rf.Roi`, `rf.Step`	The core module: `rf.lab.filter_array(array, steps)`, `rf.lab.foreground(array)`, `rf.lab.align_arrays(fixed, moving)`, `rf.lab.fit_pairs(pairs, model)` work on arrays too.

```python
with rf.open("example.rockface") as prj:
    pp, xp = prj.image("ILLUSTRATIVE mosaic PP"), prj.image("ILLUSTRATIVE mosaic XP")
    pp_c, xp_c = prj.crop([pp, xp], roi=(2.0, 2.0, 8.0, 6.0))
    better = pp_c.filter(["shading", ("brightness_contrast", {"contrast": 20}), ("unsharp", {"sigma":
↪   1.5})])
    t = xp_c.register_on(pp_c)                        # ECC from the section frames
    print(t.rotation_deg, t.scale, t.ecc)
    xp_on_pp = xp_c.resample_on(pp_c)
    mask = pp.sample_mask()
    dark = prj.calculate("{%s}.l < 60" % xp_on_pp.name, pp_c)
    print(dark.areas_mm2())
```

RockFace Lab: segmentation, map editing and objects (API 1.11, RockFace 0.18)

A segmentation is kept as a **map** in the frame of its image (category "segmentation"): phase 0 is "not classified / outside", the segments follow from 1, each with a role (`"pore"`, `"mineral"` for solid, or `"unclassified"`). Edits of a map are written into it in place and return an undo patch. Objects are

kept as an **object map** (category "objects"): every pixel holds the number of its object. Coordinates of edits are level-0 pixels of the map (the same as those of its image for a segmentation of the whole image at level 0). The methods and their checks are in SEGMENTATION.md.

Call	What it does
`img.segment(method="resin", segments=None, post=(), roi=None, save=True, name="", box=None, **params)`	Segments a picture. `method`: `"resin"` (`space="lab"` with `b_max=None` for the automatic cut and `chroma_min`, or `space="hsv"` with `hue`, `sat_min`, `val_min`), `"thresholds"` (`channel` of `rf.segment.CHANNELS`, `cuts=[…]` or `classes` for multi-level Otsu), `"colour_range"` (`ranges={"hue": (190, 250), "sat": (20, 100)}`), `"bayes"` (`polygons=[[polygons of segment 1], …]` or `samples/model`, `kind="hsv"` or `"lab"`, `texture`, `min_prob`), `"kmeans"` (`k`, `superpixels`), `"seeds"` (`strokes=[(segment 1…k, points, radius), …]` or `markers`). `segments`: names, dicts or `rf.Segment` (the resin defaults to pore and solid); `post`: `[("islands", {"min_px": 30}), ("majority", {"size": 3}), ("fill_holes", {"segment": 0}), ("margin", {"segment": 0, "px": 2})]`; `roi`: a sample mask. Returns the map, or `rf.segment.Result` (`labels`, `phases`, `porosity`, `stats`, `notes`) with `save=False`.
`img.porosity(region=None)`	Porosity (%) of a segmentation or mineral map: pore over pore and solid.
`img.paint(points, radius, phase)`	Brush along a polyline (`phase` 0: eraser).
`img.fill_polygon(polygon, phase, only=None)`	The inside of a polygon set to `phase` (only over the pixels of `only`).
`img.scissors(polygon, inside=True, phase=0)`	The inside (or outside) of a polygon set to `phase`.
`img.level_trace(x, y, tolerance, phase, source=None)`	The connected pixels within `tolerance` (ΔE*ab) of the colour at (x, y) in the picture under the map set to `phase`.
`img.remove_islands(min_px, phase=None)`	Pieces smaller than `min_px` given to their surroundings.
`img.undo(patch)`	Undoes an edit; returns the patch that redoes it.
`img.objects(phase, method="watershed", name="", **kw)`	Objects of one phase (index or name): `"islands"`, `"watershed"` (`min_distance`, `sigma`), `"snow"` (`r_max`, `sigma`); `min_px`. Returns the object map.
`img.object_count()`, `img.object_numbers(level=0)`	Number of objects; the object number of every pixel (NumPy uint32).
`img.merge_objects(points)`, `img.split_object(x, y, min_distance=5, sigma=1.0)`, `img.cut_objects(p0, p1, width=1.0)`, `img.delete_object(x, y)`	Edits of an object map, each returning an undo patch.
`rf.segment`, `rf.Segment`, `rf.Spec`	The core module: `rf.segment.run(array, spec)`, `rf.segment.partition(mask, method)`, `rf.segment.rgb_to_lab(array)`, `rf.segment.channel(array, name)` work on arrays too.

```python
with rf.open("example.rockface") as prj:
    pp = prj.image("ILLUSTRATIVE mosaic PP")
    seg = pp.segment("resin", post=[("islands", {"min_px": 20})])
    print(f"porosity {seg.porosity():.2f} %")
    km = pp.segment("kmeans", k=4, superpixels=5000, save=False)
    grains = seg.objects(2, "snow", min_px=40)
    patch = seg.paint([(100, 100), (300, 120)], 15, 1)
    seg.undo(patch)
```

RockFace Lab: measurements, flows and the whole section (API 1.12, RockFace 0.19)

A **measurement** (`rf.measure.Measurement`) holds the table of the objects of an object map (`.table`, an `rf.Table`), the throats and coordination of pores (`.throats`, `.coordination`) and gives a summary. Lengths are in µm when the map has a scale. An `rf.Table` takes computed columns

(`t.add_column("ratio", "feret_max_um / feret_min_um")`) and filters (`t.filter("area_um2 > 500 & touches_edge == 0")`), written with the column names, `{name with spaces}`, numbers, text in quotes, + − * / ** %, comparisons, & | ~ and `abs sqrt log log10 log2 exp sin cos tan degrees radians minimum maximum where clip round floor ceil`. The methods and their checks are in MEASUREMENTS.md.

Call	What it does
`img.measure(min_px=1, picture=None, throats=None, level=0)`	Measures every object of an object map: area, perimeter, Feret diameters and angle, ellipse, shape factors, centroid (px and mm), edge contact, brightness and colour of the picture (`picture`, default the one it was made from); throats and coordination for pores (or `throats=True`).
`m.summary(size="feret_max_um", by_area=False, exclude_edge=False)`	Count, Folk & Ward mean, sorting, skewness, kurtosis (φ) and their classes, median, D10/D50/D90 (mm), mean aspect ratio and circularity, mean coordination.
`rf.measure.class_table(table, size, scheme)`, `rf.measure.area_groups(table)`	Objects by Udden–Wentworth class (`"wentworth"`), pore class (`"pores"`), or groups of similar area.
`rf.measure.folk_ward(sizes_mm, weights=None)`, `rf.measure.percentile(values, weights, q)`, `rf.measure.histogram(sizes_mm)`	Distribution statistics of any sizes.
`rf.measure.chart(kind, table, path=None, x="", y="", size=..., by_area=False)`	“histogram” (φ, with the cumulative curve), “crossplot” (with marginal histograms), “rose” (an angle column), “classes”; PNG bytes, also written to `path`.
`table.to_csv(path)`, `rf.measure.to_xlsx(path, {sheet: table or dict})`	Exports.
`img.run_flow(flow="pp", xp=None, **kw)`	“pp”, “pp_xp” (XP picture `xp`, default the XP of the section) or “mineral_map”; options `spec`, `roi`, `pore_method`, `grain_method`, `min_distance`, `min_px`, `grains`, `merge_de`, `name`. Returns `rf.flows.FlowResult` (`images`, `measurements`, `porosity`, `fractions`, `sample`).
`prj.run_flow(images, flow="pp", **kw)`, `prj.flow_table(results)`, `prj.las_by_depth(results, folder, column="feret_max_um")`	A flow on several images; one row per sample; one LAS file per well (max, mean, median, min, std of the column, and the porosity).
`img.quantify_whole(method="resin", segments=None, post=(), roi=None, name="", **params)`	The whole picture segmented at full resolution block by block (pixel-by-pixel methods; majority smoothing allowed): (map, {segment: %}, porosity).
`img.import_labels(path, kind="segmentation", phases=None)`, `img.export_labels(path, level=0)`, `img.objects_png(path, max_side=4000)`	Label images in and out; the objects in colour.
`prj.export_netcdf(images, path, level=0, tables=None)`, `prj.import_netcdf(path, like)`	NetCDF (classic format) with the scale, origin and phases of every image.
`rf.measure`, `rf.flows`, `rf.Table`	The core modules: `rf.measure.measure_array(numbers, um_per_px)`, `rf.measure.throats(numbers)`, `rf.flows.grains_by_colour(mask, xp_rgb)` work on arrays too.

```python
with rf.open("example.rockface") as prj:
    pp = prj.image("ILLUSTRATIVE mosaic PP")
    r = pp.run_flow("pp")
    pores, grains = r.measurements
    print(r.porosity, grains.summary(exclude_edge=True)["d50_mm"])
    t = grains.table.filter("touches_edge == 0")
    t.add_column("ratio", "feret_max_um / feret_min_um")
    rf.measure.chart("rose", t, "orientations.png", x="ellipse_angle")
    prj.las_by_depth([r], "las")
    seg, fractions, porosity = pp.quantify_whole("resin")
```

RockFace MC: micro-CT volumes (API 1.13, RockFace 0.20)

A **volume** (`rf.VolumeAPI`) belongs to a sample and may be tied to one of its thin sections. It is grey values or segmented (labels with segments). Its voxels are kept in the project in compressed chunks with a pyramid of halved levels, so a volume larger than the memory is read a block at a time; every operation writes a new volume with its history. The methods and their checks are in MICROCT.md.

Call	What it does
`prj.import_volume(source, sample, name="", voxel_um=None, kind="grey", thin_section=None, **read)`	Imports a folder of slices, .tif, .raw (`shape=(nz, ny, nx)`, `dtype`, `byte_order`, `header`), .mhd/.mha, .nrrd/.nhdr, .nc or Imperial ASCII .dat, a NumPy array (z, y, x) or an `rf.volume.VolumeSource`; `sample` is a description (or its name) or a sample id.
`prj.import_volume_labels(source, sample, mapping, segments, ...)`	A segmented volume made elsewhere: `mapping` {value in the file: index 1…n}, `segments` names, dicts or `rf.volume.Segment`.
`prj.volumes(sample=None, kind=None)`, `prj.volume(name)`, `d.volumes()`, `d.import_volume(source, ...)`	The volumes of the project or of a description's sample (L2).
`prj.calculate_volumes(formula, {name: volume})`	The volume calculator; a condition gives a segmented 0/1 volume.
`v.shape`, `v.voxel_um`, `v.kind`, `v.phases`, `v.history`, `v.meta`, `v.update(**fields)`, `v.delete()`	The volume and its record.
`v.array(level=0)`, `v.slice(axis, index, level=0)`, `v.region(box, level=0)`, `v.source(level=0)`	Voxels as NumPy arrays (z, y, x); `source` reads slab by slab.
`v.crop(box)`, `v.resample(voxel_um=None, factor=None, order=1)`, `v.filter(*steps)`	New volumes; filters `("gaussian", {"sigma": 1})`, median, mean, unsharp, window, invert, block by block.
`v.histogram(bins=256, roi=None)`, `v.cylinder()`, `v.segment(cuts=None, classes=2, segments=None, roi=None)`	Thresholds typed or by (multi-level) Otsu; `roi` "cylinder", (cx, cy, r) or a segmented volume.
`v.porosity()`, `v.profile(axis)`, `v.connectivity(axis)`, `v.two_point(max_lag)`	Of a segmented volume.
`v.export(path)`	.mhd (+ .raw), .mha, .nrrd, .tif or .nc.
`v.register(fixed, pairs=None, model="rigid", auto=True)`, `v.resample_on(fixed)`	3D registration (landmarks and/or the automatic shift) and resampling on the fixed grid.
`img.locate_in(volume, axis="z", index_range=None, scale=None, save=True, add_slice=False)`, `v.locate(image)`, `v.slice_image(axis, index, placement=None)`	A thin-section image located in a volume (L13): slice, rotation, scale, shift (`rf.volume.Placement`); the matching slice added to the section, laid over the image.
`rf.volume`	The core module: `open_volume`, `Mapped`, `legend`, `FILTERS`, `segment_volume`, `fit_3d` and the others work on arrays and files too.

```python
import numpy as np
with rf.open("example.rockface") as prj:
    d = prj.descriptions()[0]
    ct = d.import_volume("plug.mhd")
    pores = ct.filter(("median", {"size": 3})).segment(roi="cylinder")
    print(pores.porosity()["porosity"], pores.connectivity("z")["percolates"])
    print(np.nanmean(pores.profile("z")), pores.two_point()["length_x"])
    pores.export("pores.nrrd")
    found = d.images("photo")[0].locate_in(ct, add_slice=True)
```

RockFace MC: core photographs and image logs (API 1.14, RockFace 0.21)

A **well log** (`rf.LogAPI`) belongs to a well: rows are depths (`top_m + r·step_m`), the columns of an image are azimuths clockwise from north. It is an image, labels, objects, a curve or a table by depth; every operation writes a new log. A **core** is a grey volume processed by `process_core` (its `core`

dict holds well, top, diameter, centre, rotation, depth shift). The methods and their checks are in CORE_IMAGELOG.md.

Call	What it does
`prj.import_log(path, well, diameter_mm=None, modality="other", select=None, as_table=(), top_m=, base_m=, well_diameter_mm=)`	.las, .csv / .csv.gz (matrix, Techlog, long form), .nc/.grd, .dlis (optional dlisio), a picture with its depths, or a folder of T<core>_CX<a>CX<b> unwraps with a boundaries table.
`prj.add_log(array, well, name, top_m, step_m, **fields)`	A log from an array (rows × columns; 1D for a curve; × 3 for RGB).
`prj.logs(well=None, category=None, kind=None)`, `prj.log(name)`, `prj.wells()`	The logs of the project or of a well.
`lg.array()`, `lg.depths()`, `lg.azimuths()`, `lg.read(top_m, bottom_m, max_rows)`, `lg.meta`, `lg.history`, `lg.update(**f)`, `lg.delete()`	The values (NaN where missing) and the record.
`lg.crop(top_m, bottom_m, cols)`, `lg.shift(depth_shift_m, rotation_deg)`, `lg.export(path, techlog=False)`	Crop, move and turn, export (.las, .csv, .nc, .png/.tif, .dlis).
`lg.eccentricity(transit_time, tau=None, objectives=("std",))`, `lg.spiral_filter(wl_min_m, wl_max_m)`, `lg.quality_indicator(window_m)`, `lg.heterogeneity_index(max_window_m)`, `lg.azimuth_shift(depths, degrees)`, `lg.clahe()`	Processing; `eccentricity` returns [(log, τ, value)], `spiral_filter` (log, % changed).
`lg.inpaint(mask=None, method="harmonic")`	Pad gaps and a mask filled: “linear”, “harmonic” or “biharmonic”.
`lg.segment(cuts=None, classes=2)`, `seg.objects(segment_index=1, min_px=4)`, `seg.psd()`, `psd.psd_by_depth(interval_m, bins)`	Thresholds or Otsu; objects with an `rf.Table`; local thickness and histograms by depth.
`lg.dominant_sinusoid(top_m, bottom_m)`, `lg.fit_sinusoid([(azimuth, depth), …])`	Planar features (`rf.welllog.Sinusoid`: depth, amplitude, dip, dip azimuth).
`unwrap.register_on(image_log, max_shift_m=2.0, apply=True)`	Depth shift and rotation of a core unwrap on an image log.
`prj.process_core(volume, top_m, diameter_mm, well=None)`, `prj.cores(well=None)`	Whole-core CT scans found in the liner and centred.
`prj.orient_cores(cores, "sinusoid" | "image_log", log=None)`, `v.transform_core(depth_shift_m, rotation_deg)`	Orientation and placement.
`v.unwrap(radial_depth_mm)`, `prj.well_unwrap(cores, well_diameter_mm)`, `prj.ct_log(cores)`, `v.inpaint_core(mask)`, `prj.export_cores(cores, folder)`, `prj.core_flow(volume, top_m, diameter_mm)`	Unwraps, CT log, fractures filled, export, the CoreCT flow.
`prj.load_core_photos(well, [(path or array, top_m, base_m)], rows=None, core_diameter_mm=None)`	Core-box photographs made a strip of the well.
`rf.welllog`, `rf.cores`	The core modules: `read_las`, `read_csv`, `fit_sinusoid`, `inpaint_array`, `detect_circle` and the others work on arrays and files too.

```python
with rf.open("well.rockface") as prj:
    amp, tt = prj.import_log("amplitude.las", "W-1", diameter_mm=216)[:2]
    (fixed, tau, _), = amp.eccentricity(tt)
    clean, changed = fixed.inpaint(method="biharmonic").spiral_filter(4, 20)
    print(clean.dominant_sinusoid(2501, 2510).dip_deg)
    vugs, table = clean.segment().objects(min_px=6)
    table.to_csv("vugs.csv")
```

Server mode, WITSML and OSDU (API 1.15, RockFace 0.22)

A project is a file or a **server database** (PostgreSQL). Everything above works the same on both. On a server the user is the database login, the level decides what may be written (viewer, petrographer, manager, administrator) and a description is changed only by its author or a manager — the database refuses the rest too. Script writes on a server carry the login itself (no ” [script]” tag). See SERVER.md.

Call	What it does
`rf.connect("postgresql+psycopg://user@host:5432/database")`	Open a server database (password from the pgpass file, or `rf.server.server_url(host, database, user, password)`).
`prj.is_server`, `prj.user`, `prj.role`	Where and who; a project file is its user's ("administrator").
`prj.users()`, `prj.add_user(login, password, role, full_name, institution, email)`, `prj.set_role(login, role)`	Users and levels (adding and changing: administrators).
`prj.audit(user=None, action=None, target=None, limit=None)`	Who did what, newest first.
`prj.locks()`, `prj.lock(description)`, `prj.unlock(description)`	Edit locks: `lock` returns None, or the user who holds it.
`prj.backup(path)`	The project (server or file) copied to a new project file.
`prj.publish(url, author_map=None)`	This project file copied into a new server database; authors mapped to logins.
`prj.export_witsml(folder, descriptions=None, curves=None, depth="top_depth", well_logs=True, images=True)`	WITSML 1.4.1.1: well, wellbore, the log of the samples, the logs of MC.
`prj.export_osdu(path, descriptions=None, partition="opendes", legal_tag="", owners=None, viewers=None, countries=None)`	An OSDU manifest (JSON).
`rf.server`, `rf.witsml`, `rf.osdu`	The core modules: `init_database`, `copy_project`, `changes_since`, `read_log`, `manifest`, `check`...

```python
with rf.connect("postgresql+psycopg://ana@lab-server:5432/rockface") as prj:
    print(prj.user, prj.role, len(prj.descriptions()))
    holder = prj.lock("ILL-C001")
    if holder is None:
        ...                                   # edit, save, then:
        prj.unlock("ILL-C001")
    prj.export_witsml("witsml")
```

Random forest, volume tools, log editing (API 1.16, RockFace 0.23)

Call	What it does
`img.forest(strokes=None, markers=None, segments=None, preset="balanced", trees=32, extra=(), box=None, model=None, save=True, name="")`	Random forest segmentation of a picture from painted strokes `[(segment, points, radius px)]` or a marker array (0 = not painted); `extra` pictures add channels; `model` applies a kept forest. Returns (map or `Result`, forest).
`img.apply_forest(model, level=None, extra=(), roi=None, name="")`	A forest applied to the whole picture block by block: (map, {segment: %}, porosity).
`img.transitions(percent=True, include_background=False)`	The transitions between the phases of a map (`rf.Table`: "phase" and one column per phase).
`prj.models(spatial=None)`, `prj.model(id or name)`, `prj.import_model(path, name="")`, `prj.export_model(model, path)`	The forests and Bayesian models kept in the project (data only, `.rfmodel` files).
`vol.new_annotation(segments=("Pore", "Solid"), name="")`	An empty labels volume on the grid of a volume, to paint on.
`ann.paint(axis, index, points, radius, value, ball=False, only=None)`, `ann.undo(patch)`	A brush stroke on a slice (`points` [(h, v)] voxels of the slice); 0 erases.
`vol.train_forest(annotation, preset="balanced", trees=32, extra=(), name="", method="forest")`	A forest (or `method="bayes"`, a Gaussian model) from the painted voxels; kept when `name` is given.
`vol.apply_forest(model, extra=(), roi=None, name="")`	The whole volume classified block by block (fractions and porosity in `.meta`).
`seg.edit(effect, **params)`	Segment-editor effects on a labels volume: margin, smooth, islands, logical, mask, expand, boundary_removal, grow_from_seeds, scissors, level_trace (`rf.vol_tools.edit_labels`).
`seg.partition(phases=None, method="watershed", sigma=1.0, min_distance=4, min_voxels=1)`, `objects.objects()`	The pore space split into objects (an instances volume) and their table.

Call	What it does
`vol.filter(("anisotropic", {"conductance": 1.0, "iterations": 5, "time_step": 0.0625}))`	Anisotropic diffusion (edge-preserving), block by block.
`vol.shading_correction(mask=None, points=2000, smooth=0.0)`	The shading corrected slice by slice (`mask`: None, "cylinder" or a labels volume).
`vol.register_rigid(fixed, save=True)`	The rotation and shift found without landmarks; `rf.vol_tools.rotation_angles(t.matrix)`.
`vol.variogram(phases=None, roi=None, lags=30)`, `vol.rev(...)`, `vol.variogram_report(path)`	Semivariogram along x, y, z and all directions with the spherical model; REV; HTML report.
`vol.attributes()`, `vol.import_pcr(path)`	The attributes of a volume; a `.pcr` reconstruction file kept in them.
`log.paint(points, radius_mm, value, only=None)`, `log.undo(patch)`	Brush on a labels or objects log (`points` [(azimuth °, depth m)]).
`objs.instances()`, `objs.decline(labels)`, `objs.merge(labels)`, `objs.keep_only(condition)`, `objs.paint_instance(points, radius_mm, label=None, erase=False)`	The editor of the objects of a log.
`rf.rforest`, `rf.vol_tools`, `rf.log_edit`	The core modules: `features`, `train`, `predict`, `Forest`, `Gaussian`, `segment_array`, `edit_labels`, `auto_rigid`…

```
pp = prj.image("ILLUSTRATIVE mosaic PP")
seg, forest = pp.forest(strokes=[(1, [(120, 80)], 6), (2, [(600, 420)], 6)], segments=["Pore",
↪  "Solid"])
whole, fractions, porosity = pp.apply_forest(forest)
print(whole.transitions().rows())
vol = prj.volume("Plug A")
ann = vol.new_annotation(["Vug", "Matrix"])
ann.paint("z", 40, [(30, 30), (36, 30)], 2.0, 1)
ann.paint("z", 40, [(5, 5), (5, 60)], 2.0, 2)
vugs = vol.apply_forest(vol.train_forest(ann, name="Vugs"))
objects = vugs.partition()
print(len(objects.objects()), vugs.variogram()["r"]["range"])
```

Pore networks and open rock data (API 1.17, RockFace 0.24)

Call	What it does
`seg.extract_network(phases=None, sigma=0.4, r_max=4, name="", level=None, roi=None, keep_regions=True)`	The pore network of a segmented (its pore segments) or objects volume, by SNOW; kept in the project, the regions of its pores as an objects volume. Returns an `rf.NetworkAPI`.
`prj.networks(sample=None)`, `prj.network(name or id)`	The networks of the project.
`prj.import_statoil(path, sample=None, name="")`, `net.export_statoil(path, axis="x")`	The Statoil files (`*_node1.dat`, `*_node2.dat`, `*_link1.dat`, `*_link2.dat`) read and written.
`net.stats()`, `net.raw`	Pores, throats, coordination, isolated pores, radii (µm), porosity; the `rf.pnm.Network` (numpy arrays).
`net.kabs(axes=("x", "y", "z"), mu=1e-3)`	Absolute permeability along the axes: `{"x": {"K_mD", "K_m2", …}, …, "mean_mD"}`.
`net.micp(sigma=0.485, theta=140.0, points=60)`	Mercury intrusion from every face: `P_Pa`, `P_psi`, `S_Hg`, `r_um`.
`net.two_phase(keep_states=False, **params)`	Drainage then imbibition (`rf.pnm.TWO_PHASE`: `sigma`, `mu_w`, `mu_o`, `theta_r`, `theta_r_range`, `separation` or `theta_a`, `sw_step`, `swi_min`, `pc_max`, `axis`, `seed`, `box_lower`, `box_upper`): `rows` (cycle, Sw, Pc_Pa, krw, kro), `Swi`, `Sor`, `kro_Swi`, `krw_Sor`, `crossover_Sw`.
`net.sensitivity(grid, **base)`, `rf.pnm.curve_cloud(runs)`, `rf.pnm.correlations(runs)`	Two-phase runs over every combination of `grid` ({parameter: [values]}); the curve cloud and the parameter–result correlations.
`net.production(result=None, mu_w=1e-3, mu_o=1e-3, smoothing=1)`	Buckley–Leverett: `Swf`, `PVI_bt`, `PVI`, `Np`, `RF`, `water_cut`; a list of runs gives `pessimistic`, `neutral`, `optimistic`. `rf.pnm.recovery_at(prod, pvi)`.

Call	What it does
`seg.kabs_rev(lengths=5, min_fraction=0.3)`	Kabs of centred cubes of growing edge, each extracted again.
`net.report(path)`, `net.results`, `net.save(path)`, `net.delete()`	HTML report of the network and its kept results; the results kept with it; the network as a file.
`rf.ProjectAPI.rock_data()`, `prj.download_rock_data(entry_id, sample, folder=None, name="")`	The catalogue of public digital-rock images (`rf.rock_data.Entry`) and one downloaded and imported with its licence and citation.
`rf.pnm`, `rf.rock_data`	The core modules: `extract_array`, `read_statoil`, `kabs`, `micp`, `two_phase`, `production`, `kabs_rev`, `report`, `catalogue`…

```
seg = prj.volume("ILLUSTRATIVE sphere pack")
net = seg.extract_network(name="ILLUSTRATIVE network")
print(net.kabs()["mean_mD"], net.stats()["coordination_mean"])
kr = net.two_phase(theta_r=30, separation=20, swi_min=0.2)
print(rf.pnm.recovery_at(net.production(kr, mu_o=5e-3), 2.0))
```

Microporosity, Microtom, multiscale, surfaces, Geolog and jobs (API 1.18, RockFace 0.25)

Call	What it does
`grey.porosity_map(labels, classes=None, i_pore=None, i_solid=None, saturated=None, name="")`	The porosity map (0 solid … 1 resolved pore) of a grey volume from its segmentation (`classes` {index: "macro" | "micro" | "solid" | "high"}), or of a dry scan and the `saturated` one. Returns (`VolumeAPI`, info with `porosity`, `macro`, `micro` in %, the attenuations used).
`phi.darcy(axes=("x", "y", "z"), model="power", a_md=1.0, b=3.0, grain_um=10.0, k_macro_md=None)`	Darcy permeability of a porosity map: `{"x": {"K_mD", …}, …, "mean_mD"}` (`rf.microporosity.MODELS`).
`seg.microtom(method, **kw)`	A Microtom simulation (`rf.microtom.METHODS`: `psd`, `hierarchical_psd`, `micp`, `drainage`, `imbibition`, `imbibition_trapped`, `permeability`, `permeability_rev`); options `direction`, `resolution`, `sigma`, `theta`, `tau`, `tol`, `max_steps`, `max_voxels`, `axes`. A map made is in `result["map"]`.
`seg.lbm_permeability(axes=("x", "y", "z"), **kw)`	Absolute permeability by lattice Boltzmann (TRT, D3Q19): per axis `K_mD`, `K_m2`, `steps`, `converged`; `mean_mD`.
`grey.modelling_flow(crop=None, roi_id=None, median=0, cuts=None, segments=None, boundary=None, classes=None)`	Crop, ROI, median, thresholds, boundary removal and porosity map in one call; the ids of every volume made.
`seg.mps(realizations=1, shape=None, hard=None, log=None, seed=1, n_cond=24, radius=12, threshold=0.05, fraction=0.3)`	MPS realizations (direct sampling) with this volume as the training image, conditioned by a hard-data volume or an image log (`LogAPI`) wrapped on a cylinder; a list of `VolumeAPI`. `rf.mps.porosity_per_slice`, `rf.mps.psd_compare`, `rf.mps.two_point` compare them.
`phi.multiscale_network(subscale=None, tortuosity=2.0, micro_block=6, macro=0.99, name="")`	The two-scale pore network of a porosity map; `subscale` {"kind": "fixed", "radius_um"} | {"kind": "leverett", "k_md"} | {"kind": "micp", "P_Pa", "S"}. Returns an `rf.NetworkAPI`.
`vol.save_into_netcdf(path, overwrite=False, level=0)`	The volume saved as a new variable of an existing NetCDF file, which keeps what it holds; `{name: variable}`.
`seg.surface(segment=1, name="")`, `prj.import_surface(path, sample=None, scale_mm=1.0)`, `prj.surfaces(sample=None)`	3D surfaces (`rf.SurfaceAPI`): of a segment (marching cubes), or read from OBJ with MTL and textures.
`s.mesh`, `s.export_obj(path)`, `s.render(path=None, width, height, yaw, pitch)`, `s.delete()`	The `rf.surfaces.Mesh`; OBJ + MTL + PNG; a picture.
`log.permeability_model(porosity, plugs, macro=None, ignore=(), kds=None, window_m=0.0)`	The permeability curve of a segmented image log fitted to the plugs [(depth m, K mD)] (Menezes de Jesus et al. 2016); returns (`LogAPI`, fit with `A`, `B`, `rms_log10`, `r2_log10`). `rf.permeability_model.fit` and `.model` on arrays.

Call	What it does
`prj.geolog_export(well, folder, logs=None, null=-999.25)`, `prj.geolog_import(path, well="", nulls=…, diameter_mm=None)`	The Geolog package (LAS, DLIS when `dliswriter` is installed, manifest) and Geolog's files read back.
`prj.prepare_job(function, folder=None, name="", **kwargs)`, `prj.batch(function, over, ids, folder=None, **kwargs)`	Job folders for one of `rf.jobs.FUNCTIONS` (inputs copied into a small project); one per id for a batch.
`prj.run_jobs(jobs, workers=2, wait=True)`, `prj.bring_back(job)`	Run them in other processes; copy what a finished job made into the project.
`rf.jobs.Host(host, workdir, python, pythonpath, setup, scheduler="none" | "slurm", slurm, port)`, `rf.jobs.RemoteRunner(host).submit(job)`, `.poll(job)`, `.fetch(job)`, `.cancel(job)`, `.script(job)`	Jobs on a remote computer by SSH and scp (the user's keys), in the background or with `sbatch`.
`rf.microporosity`, `rf.microtom`, `rf.mps`, `rf.surfaces`, `rf.geolog`, `rf.jobs`, `rf.permeability_model`	The core modules.

```
grey, seg = prj.volume("ILLUSTRATIVE grey"), prj.volume("ILLUSTRATIVE segmentation")
phi, info = grey.porosity_map(seg, {1: "macro", 2: "micro", 3: "solid"})
print(info["porosity"], phi.darcy()["mean_mD"], seg.lbm_permeability(axes=("x",))["x"]["K_mD"])
net = phi.multiscale_network({"kind": "fixed", "radius_um": 0.5})
job = prj.prepare_job("microtom", volume_id=seg.id, method="permeability")
prj.run_jobs([job])
print(prj.bring_back(job))
```

RockFace AI (API 1.19, RockFace 0.26)

Applying models needs the extra `ai` (ONNX Runtime); training needs `ai-train` (PyTorch). Without them these calls raise `rf.ScriptError` with the message saying what to install; everything else works. A model is named by its name (the newest version), an entry of `prj.ai_models()`, the id of its record in the project, a `.rfai` path, or an `rf.ai.hub.Model`.

Call	What it does
`prj.ai_hardware()`	`{"hardware": {…}, "options": [{key, device, packages, size, recommended, available, install}], "device", "inference", "training"}`
`prj.ai_models(task=None, folder=None)`	Every model: `source` (builtin, hub, project), `ref`, `name`, `version`, `task` (`rf.ai.hub.TASKS`), `licence`, `parent`, `metrics` (held out), `description`.
`prj.ai_model(which, version=None)`	The `rf.ai.hub.Model` (`.card`, `.onnx`, `.run(x)`).
`prj.ai_import(path)`, `prj.ai_export(model, path)`, `prj.ai_publish(model, folder=None)`, `prj.ai_delete(id)`, `prj.ai_lineage(model)`	The hub: `.rfai` files, the hub folder (`<name>/<version>.rfai` and `index.json`, never overwritten), the model and its parents.
`img.ai_segment(model, name="", level=None)`	The picture (whole section, block by block) segmented: (map `ImageAPI`, {class: %}, porosity, mean confidence).
`img.ai_train(strokes=None, map=None, segments=None, base=None, name="", size="standard", epochs=20, steps=40, holdout=0.2, seed=0)`	Train (or fine-tune `base`) on strokes [(class, points px, radius px)] named by `segments`, or on a segmentation `map` of the picture; returns (model kept in the project, held-out metrics: `accuracy`, `mean_iou`, `mean_dice`, `per_class`, `confusion`).
`img.ai_validate(model, map)`	The metrics against a reference map (classes matched by name).
`img.ai_objects(model, min_px=6, name="")`, `img.ai_train_objects(objects, base=None, name="", size="standard", epochs=20, steps=40, holdout=0.2)`	Grains or pores as separate objects by an instance model (object map, count); an instance model trained or fine-tuned on an object map of the picture (model, metrics with the object F1, precision and recall).
`img.ai_unsupervised(max_segments=32, min_segments=3, iterations=200, max_px=1_000_000, name="", seed=0)`	Segmentation without annotation: (map, {segment: %}).
`vol.ai_segment(model, name="", level=None)`	A grey volume segmented (3D, or a 2D model slice by slice): (labels `VolumeAPI`, {class: %}, porosity, confidence).

Call	What it does
`vol.ai_train(labels, base=None, dims=2, name="", size="standard", epochs=20, steps=40, holdout=0.2, level=0)`, `vol.ai_validate(model, labels)`	Train or fine-tune on an annotation or segmented volume of the same grid (held-out slices in 2D, blocks in 3D); validate.
`vol.ai_generator(iterations=500, max_voxels=64**3, width=32, name="")`, `vol.ai_generate(model, n=1, scale=1.0, seed=1)`	SinGAN trained on the volume (categorical when segmented); returns (model, comparison of 4 generated volumes); new volumes in the sample.
`log.ai_segment(model, name="")`, `amp.ai_instances(model, transit_time=None, min_px=6)`	An image log segmented (labels log, {class: %}, confidence); objects by the instance segmenter (object log, table, count).
`prj.ai_spots_train(kind=None, thin_section=None, elements=None, folds=5)`, `prj.ai_spots_propose(id, kind=None, thin_section=None, min_probability=0)`, `prj.ai_spots_accept(proposals, id)`	The spot cycle: (record id, metrics by folds of thin sections); proposals {spot_id, name, constituent, probability}; accepted into the spots.
`rf.ai.hardware`, `rf.ai.hub`, `rf.ai.infer`, `rf.ai.train`, `rf.ai.cycle`, `rf.ai.unsupervised`, `rf.ai.instances`, `rf.ai.singan`, `rf.ai.spots_ml`, `rf.ai.synthetic`	The core modules (arrays in, arrays out); `rf.ai.synthetic.microct_volume(shape, seed)` and `rf.ai.instances.synthetic_log(seed=…)` make ILLUSTRATIVE data of exact truth.

```
pic = prj.image("ILLUSTRATIVE mosaic PP")
seg, fr, por, conf = pic.ai_segment("rockface-thinsection-pores")
grey, truth = rf.ai.synthetic.microct_volume((24, 64, 64), seed=1)
g = prj.import_volume(grey, prj.descriptions()[0], name="ILLUSTRATIVE scan", voxel_um=5.0)
labels, fr, por, conf = g.ai_segment("rockface-microct-pores")
```

Mio and the local language model (API 1.20, RockFace 0.27)

Call	What it does
`prj.ask(question, model=None, lang="", use_model="auto")`	The descriptions that answer a question in plain language (L1): an `Answer` with `names`, `ids`, `count`, `explanation` (the conditions read, in words), `unknown` (words not used), `method` (“rules” or “model”), `rewritten` (the model’s reading) and `query()` (the Explorer query, when it is one). `model=True` loads the model configured
`prj.mio(model=None, lang="")`	Mio on the project: `.ask("@mio …", polish=False)` → a `Reply` with `tool`, `args`, `data`, `text`, `method`, `steps` (a skill’s replies)
`prj.mio_skills(include_inactive=False)`	Mio’s skills: RockFace’s, the user’s folder’s and the project’s (`rf.skills.Skill`: `name`, `title`, `triggers`, `inputs`, `steps`, `active`, `source`)
`prj.save_mio_skill(dict)`, `prj.delete_mio_skill(name)`	A skill kept in the project (checked first; `ScriptError` with the problems)
`prj.suggest_description(map, objects=None, region=None, points=300)`	A segmentation map (image, id or name) turned into suggestions (`rf.suggest.Suggestion`: `items` with `key`, `percent`, `points`, `constituent`, `kind`, `alternatives`, `applicable`); with an object map of grains, grain size and sorting too
`d.suggestion(map=None, objects=None)`, `d.apply_suggestion(s, accept=None)`	The suggestion of the last map of a description’s thin section; the accepted items written (keys; by default every applicable one)
`rf.llm.recommend()`, `rf.llm.CATALOGUE`, `rf.llm.download(key, progress=…)`	The models that fit this computer (recommended one marked), the catalogue (licence, size), a download with its SHA-256
`rf.llm.load()`, `rf.llm.LlamaModel(path, device="cpu"|"gpu")`, `rf.llm.ServerModel(url, model)`	A language model: the one configured, a GGUF file run by llama.cpp, or a local OpenAI-compatible server; `.complete_json(prompt, schema)`, `.chat(messages)`
`rf.nlq.ask(explorer, question, model)`, `rf.nlq_bench.build(path)`, `rf.suggest.text(s, lang, kb)`	The search itself, the ILLUSTRATIVE test set of 0.27, the template sentence of a suggestion

Parity closure (API 1.21, RockFace 0.28)

Call	What it does
`d.classify("mcbride1963")`, `d.classify("bramkamp_powers1958")`, `d.symbol(code="bramkamp_powers1958")`	McBride (1963) and Bramkamp & Powers (1958); the reservoir-rock symbol (e.g. “21BD”, “?” while the porosity A/B/C is not chosen). The carbonate inputs: `d.carbonate.bp_texture`, `bp_porosity`, `bp_agent`, `bp_facies`, `bp_group`, `bp_special`, `dolomite_pct`
`d.packing()`, `rf.packing_class(pp)`	Kahn’s packing proximity and density and the class (loose < 40 %, normal 40–55 %, tight > 55 %); `rf.packing_help.figure(lang)` the three ILLUSTRATIVE packings
`rf.volume.open_volume(path)`, `rf.formats.read_amira`, `write_amira`, `HDF5Source`, `DicomSeries`, `read_tescan`, `rf.geoslicer.read_seg_nrrd`, `write_seg_nrrd`, `import_scene`	The volume formats of the reference programs (Amira, HDF5, DICOM, TESCAN, GeoSlicer scenes and segmentations); `rf.volume.export(prj.raw, id, "x.am" | "x.h5" | "x.seg.nrrd")`
`prj.import_lab_report(path, well, tolerance_m=0.10, laboratory="")`, `rf.pdf_tables.read_tables(path)`	Tables of a lab report in PDF as routine core analyses of the samples at those depths
`rf.atlas.read_scale_bar(prj.raw, image_id, length_um=None)`, `rf.scale_bar.find_bar`, `read_label`	The scale bar of a picture found and read (Tesseract) and its µm per pixel
`prj.composite_3d(map, volume, axis="z", index=None, matrix=None)`, `rf.composite3d.property_volume(...)`	The mineral map carried to the whole volume (held-out agreement in `held_out`); a volume of a property per phase (L13)
`prj.publication(**changes)`, `prj.publish(force=False, token=None)`, `rf.publish.Client`	The periodic publication (folder or OSDU RAFS DDMS): settings saved; only what changed sent. `prj.publish(url)` with a server address is still the server copy of 0.22
`prj.survey(well, rows_or_path, datum_elevation_m=None, north="true")`, `rf.trajectory.sample_position(prj.raw, sample)`	A directional survey kept on a well (minimum curvature); a sample’s TVD, TVDSS, offsets and latitude–longitude
`rf.plugins.install_zip(path, sha256="")`, `install_git(url, ref="", expected_commit="")`, `install_from_server(name, url="")`, `enable`, `remove`, `check_update(url="")`	Plug-ins and the update check
`rf.catalogue.FolderConnector(root)`, `OsduConnector(url, partition, token)`, `import_items(prj.raw, connector, sample, items)`	The sample catalogue and its import
`prj.touchstone_table(path, names=None, mapping=None)`, `prj.import_mapped_xml(path, mapping, on_conflict="rename")`	The Touchstone-ready table; descriptions read from any XML through a mapping (`rf.interop.EXAMPLE_MAPPING`)
`rf.gs_tools.cups`, `remap`, `distance_map`, `FolderWatch`, `reconstruct_skyscan`, `reconstruct_fdk`, `reconstruct_parallel`, `upscale_to_las`, `plug_to_core`; `rf.log_edit.fill_depth_range`, `level_trace`	The tools of the GeoSlicer source audit
`prj.lbpm(labels, roles, folder=None, run=True, nproc=(1, 1, 1), affinities=None, params=None, program="lbpm_color_simulator")`, `rf.lbpm.write_case`, `command`, `slurm_script`, `read_results`, `figure`	Two-phase flow with the user’s LBPM: the case, the run (blocking; or the job `lbpm`), the curves and end points
`img.register_on(fixed)` on two maps, `rf.lab.register_map(prj.raw, fixed_id, map_id, fixed_phases=None, map_phases=None)`	A mineral map aligned on a segmentation without landmarks (their pores)
`rf.vol_tools.edit_labels(prj.raw, id, effect, blockwise=False, **p)`	Margin, smoothing, boundary removal and expand at full resolution slab by slab (automatic on big volumes)

The user guide and the feedback form (API 1.22, RockFace 0.29)

Call	What it does
`d.add_analysis(kind, laboratory="", date="", values=None, rows=None, texts=None, notes="", pictures=())`	An additional analysis of the description's sample (P15), written at once; `kind` one of `rf.analyses.KINDS` (conventional and special petrophysics, XRD whole rock and clay fraction, SEM SE and BSE, stable and Sr isotopes, chemistry, biostratigraphy); pictures copied into the project; `ScriptError` on a value out of range or a bad date
`d.analyses(kind=None)`, `d.delete_analysis(id)`	The analyses of the sample, seen from every description of it (dicts: id, kind, laboratory, date, values, rows, texts, notes, pictures)
`d.add_request(kind, objective, status="requested", notes="")`, `d.requests(status=None)`, `d.set_request_status(id, status)`	Analysis requests of the sample (P16): the objective is required; status requested, in_progress, done or cancelled
`rf.feedback_url()`, `rf.send_feedback(program="petro")`	The address of the testers' feedback form ("" while it is not defined), and the form opened in the browser (O2); the address comes from `ROCKFACE_FEEDBACK_URL`, then the preference `feedback/url`. In a terminal: `rockface feedback [--program lab] [--no-open]`

The user guide (`docs/guide/<en|pt|es>/`) has a **From a script** section on every page with the calls of that function.

Mio's connectors, Mio's scripts and the MCP server (API 1.23, RockFace 0.30)

Call	What it does
`rf.connector_catalogue()`	The connectors RockFace knows: key, name, group (google, microsoft, references, geodata, dev), kind (builtin or mcp), how an MCP server starts (`command`) or where it is (`url`), the keys (`secrets`) and settings it needs, its documentation
`prj.connectors()`	Every connector with `on` (switched on for this project) and the keys still `missing` on this computer; the project's own MCP servers too
`prj.enable_connector(key, on=True)`, `prj.disable_connector(key)`	Switch a connector on or off for this project (all are off until switched on)
`prj.add_connector(key, name, command=[…] or url="https://…", secrets=(), text="")`	An MCP server of your choice for this project (starts off)
`rf.set_connector_secret(key, name, value)`	A key or token kept only on this computer (`~/.rockface/connectors.json`, or `ROCKFACE_CONNECTORS_FILE`; environment `ROCKFACE_<KEY>_<NAME>` wins); never in a project; an empty value removes it
`prj.connector_setting(key, name, value=None)`	A plain setting kept in the project: `("zotero", "user_id")`, `("github", "repo")`, `("crossref", "mailto")`
`prj.set_confidential(on=True)`, `prj.is_confidential()`	A confidential project refuses every call that sends project data out unless confirmed twice (`allow_confidential=True`)
`prj.connector_tools(key)`	The tools of a connector: name, description, JSON schema of the arguments (an MCP server is started and asked)
`prj.connector_call(key, tool, args, confirm=…, allow_confidential=False)`	One call: `confirm=True` when you wrote it yourself, or a function that gets the `Call` (`call.preview()`) and returns True; returns `rf.connectors.Result` (`ok`, `data`, `text`, `error`, `seconds`)
`prj.connector_plan(key, action, payload)`, `prj.connector_run(call, confirm=…)`	Mio's actions on an MCP server — `export_table` (`title`, `rows`), `write_document` (`title`, `text`), `create_presentation`, `draft_email` (`to`, `subject`, `text`), `create_event` (`title`, `start`, `end`, `text`), `create_task` (`title`, `text`), `upload_file` (`path`, `title`) — matched to the server's tool (never a deleting one) and its parameters; run after confirmation

Call	What it does
`prj.connector_calls()`	The calls in the audit: when, user, connector.tool, outcome (done, declined, refused, failed) and the digest of the arguments
`rf.connectors.reference_text(item)`	A Crossref item as a line of a reference list
`prj.mio_script(request, lang="", recipe=None, **params)`	The script Mio writes for a task: `Script` (`recipe`, `code`, `params`, `criteria`, `explanation`); the samples of the request become Explorer criteria (`CRITERIA`)
`rf.script_recipes()`	The fourteen tasks Mio writes scripts for, with their parameters
`rf.mcp_config(path, allow_scripts=False)`, `rf.api_reference()`	The `mcpServers` entry for an assistant; this API in text (the server's `api` tool)

Built-in tools: Crossref `search_works(query, rows)`, `get_work(doi)`; Macrostrat `strat_names(name)`, `units(strat_name, lat, lng)`; Mindat `mineral(name)`; Zotero `search(query, limit)`, `add_item(item)`; GitHub `create_issue(repo, title, body, labels)`, `search_issues(repo, query)`. In Mio's panel: `@mio …` requests become a prepared call (shown, then run on confirmation) or a script (opened in the script window). In a terminal: `rockface connectors list|enable|disable|test|tools|secret|call|confidential …` and `rockface mcp serve project.rockface [--allow-scripts] [--http PORT] [--lang en|pt|es]`, `rockface mcp config …`.

Figures

`rf.ternary_figure(path, scheme, descriptions, original=False, width=2000, colours=True, names=True)` draws a ternary classification with the points of several descriptions and saves it as PNG or SVG. It works without the interface: an off-screen Qt is started when needed.

Records and classes

`rf.Identification`, `rf.Macro`, `rf.Micro`, `rf.Mode`, `rf.Structure`, `rf.Contact`, `rf.PoreSystem`, `rf.Pore`, `rf.Composition`, `rf.Line`, `rf.Selector`, `rf.TotalDef`, `rf.CarbonateInputs`, `rf.Scheme`, `rf.Pole`, `rf.Field`, `rf.Analysis`, `rf.Request`. The core modules are reachable as `rf.cl` (classification), `rf.co` (composition), `rf.ct` (counting), `rf.tt` (totals) and `rf.tx` (texture).

Glossário

Afloramento — local de coleta em superfície; no RockFace, um "poço" do tipo afloramento.

Amostra — o pedaço de rocha de onde sai a lâmina: plugue, testemunho inteiro, amostra lateral, calha ou de afloramento, com as suas profundidades.

Análise pontual — uma medida feita num ponto da lâmina (geoquímica, datação U-Pb, inclusões fluidas), guardada no Atlas com a sua posição e a foto da área.

Auditoria — o registro das mudanças do projeto (quem, quando, o quê), inclusive as chamadas dos conectores e do servidor MCP.

Composição original — a composição reconstruída antes da diagênese: o que foi substituído ou dissolvido volta ao constituinte de origem.

Conector — a ligação da Mio a um serviço externo (Google, Microsoft 365, Crossref…), desligada até ser ligada no projeto.

Descrição — o trabalho de um petrógrafo sobre uma lâmina; uma lâmina pode ter várias.

Grade — os pontos da contagem, em milímetros na lâmina.

ILUSTRATIVO — dado inventado para exemplos e testes, sempre marcado ILLUSTRATIVE.

Lâmina — a lâmina delgada; é ela que tem o referencial das coordenadas.

Linha de composição — um constituinte com os seus atributos e a sua quantidade.

MCP (Model Context Protocol) — o padrão aberto com que assistentes de IA falam com serviços e programas.

Mio — a assistente do RockFace: responde perguntas em linguagem natural, executa habilidades, escreve scripts e prepara chamadas aos conectores.

Modelo (template) — uma descrição-base para começar outras.

Nomenclatura — os termos do RockFace (constituintes, hábitos, localizações, modificadores, relações, poros…), cada um com a sua fonte, em três idiomas; o projeto pode acrescentar os seus.

Projeto — o arquivo `.rockface` (ou o banco do servidor) que guarda tudo.

Referencial da lâmina — o sistema de coordenadas em milímetros de uma lâmina, comum a todos os programas.

Regime diagenético — eodiagênese, mesodiagênese ou telodiagênese.

Script — um programa em Python que usa a interface de programação do RockFace (`rf`).

Servidor — o banco PostgreSQL compartilhado por uma equipe, com níveis de usuário e auditoria.